Oct 23, 2023

# You Cannot Build Large Data Projects Until You Learn Data Version Control!

The underappreciated, yet critical, skill that most data scientists overlook.

Avi Chawla

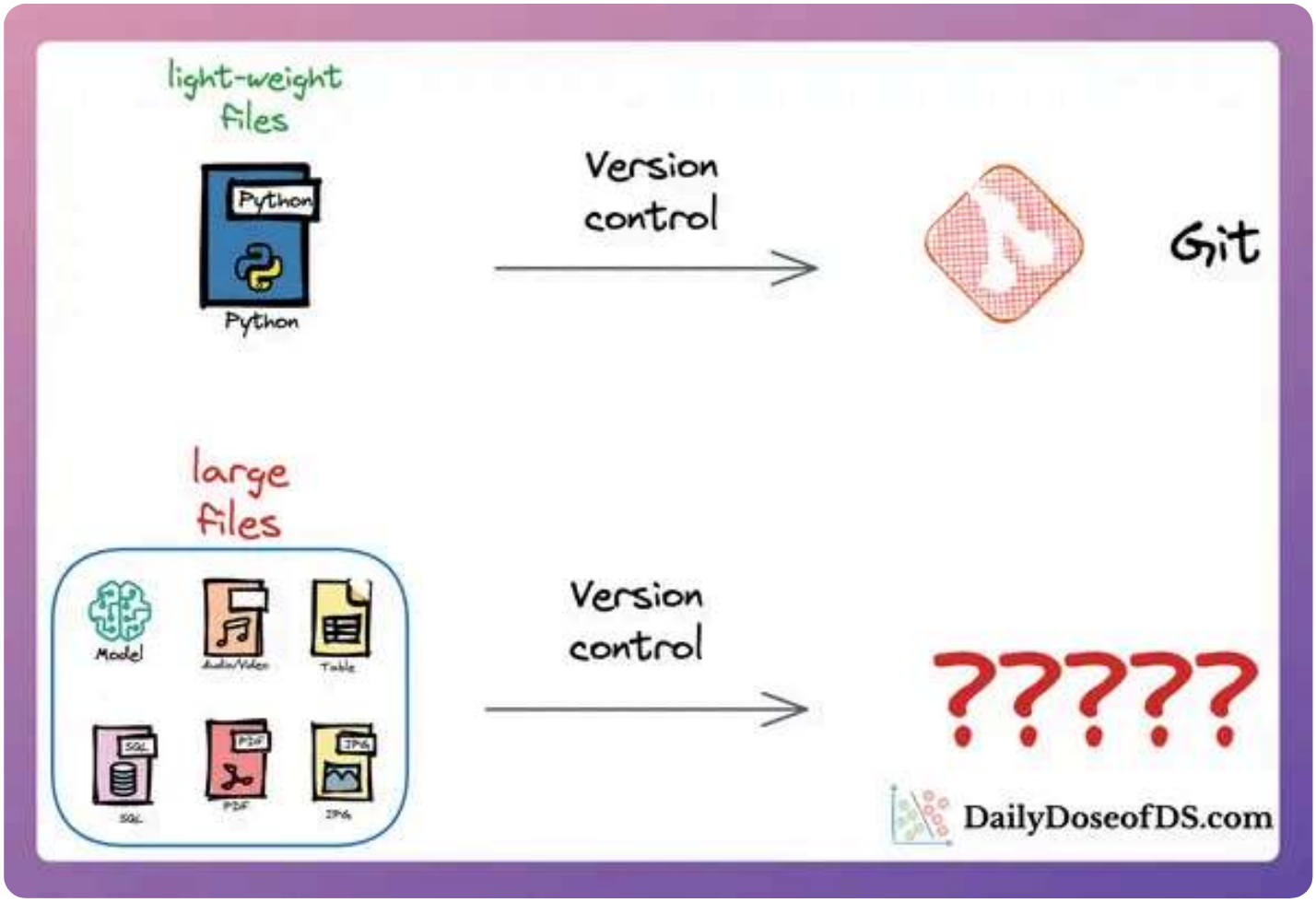
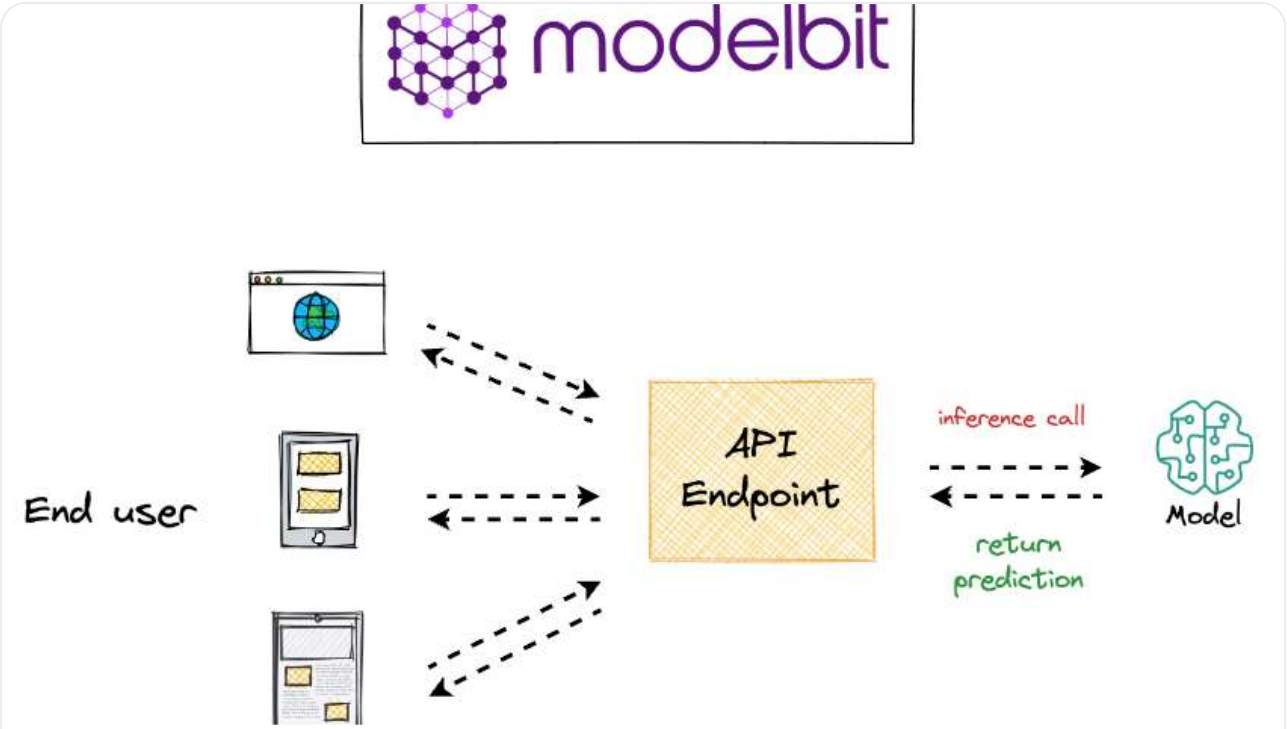



TABLE OF CONTENTS

# Recap

In a recent deep dive into model deployment, we discussed the importance of version-controlling deployments in machine learning (ML) projects:




**Deploy, Version Control, and Manage ML Models Right From Your Jupyter Notebook with Modelbit**

Deployment has possibly never been so simple.

Daily Dose of Data Science • Avi Chawla

More specifically, we looked at techniques to version control:

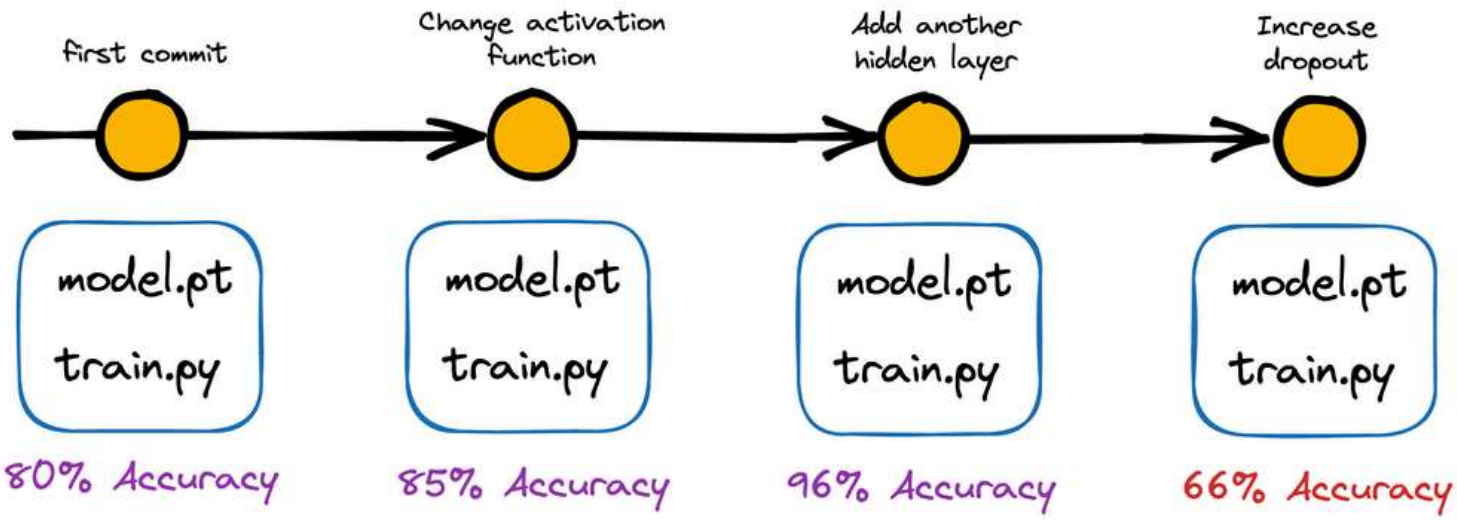
- Our deployment code, and
- Our deployed model.

Moving on, we also looked at various advantages of version-controlling model deployments.

Let's recap those.

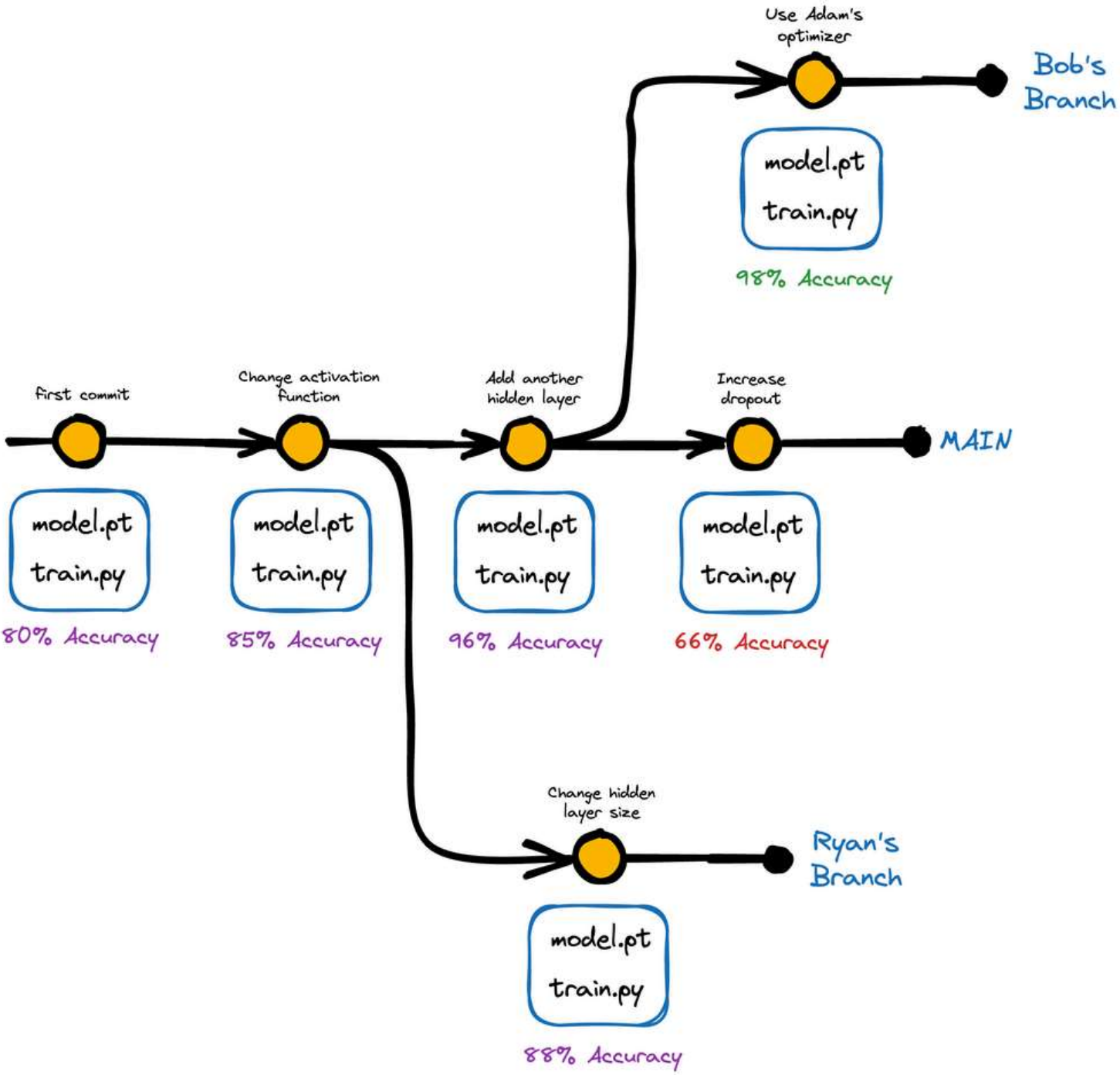
## Benefits of version control

For instance, with version control, one can precisely identify what changed, when it changed, and who changed it — which is crucial information when trying to diagnose and fix issues that arise during the deployment process or if models start underperforming post-deployment.




**Another advantage of version control is effective collaboration.**

For instance, someone in the team might be working on identifying better features for the model, and someone else might be responsible for fine-tuning hyperparameters or optimizing the deployment infrastructure.

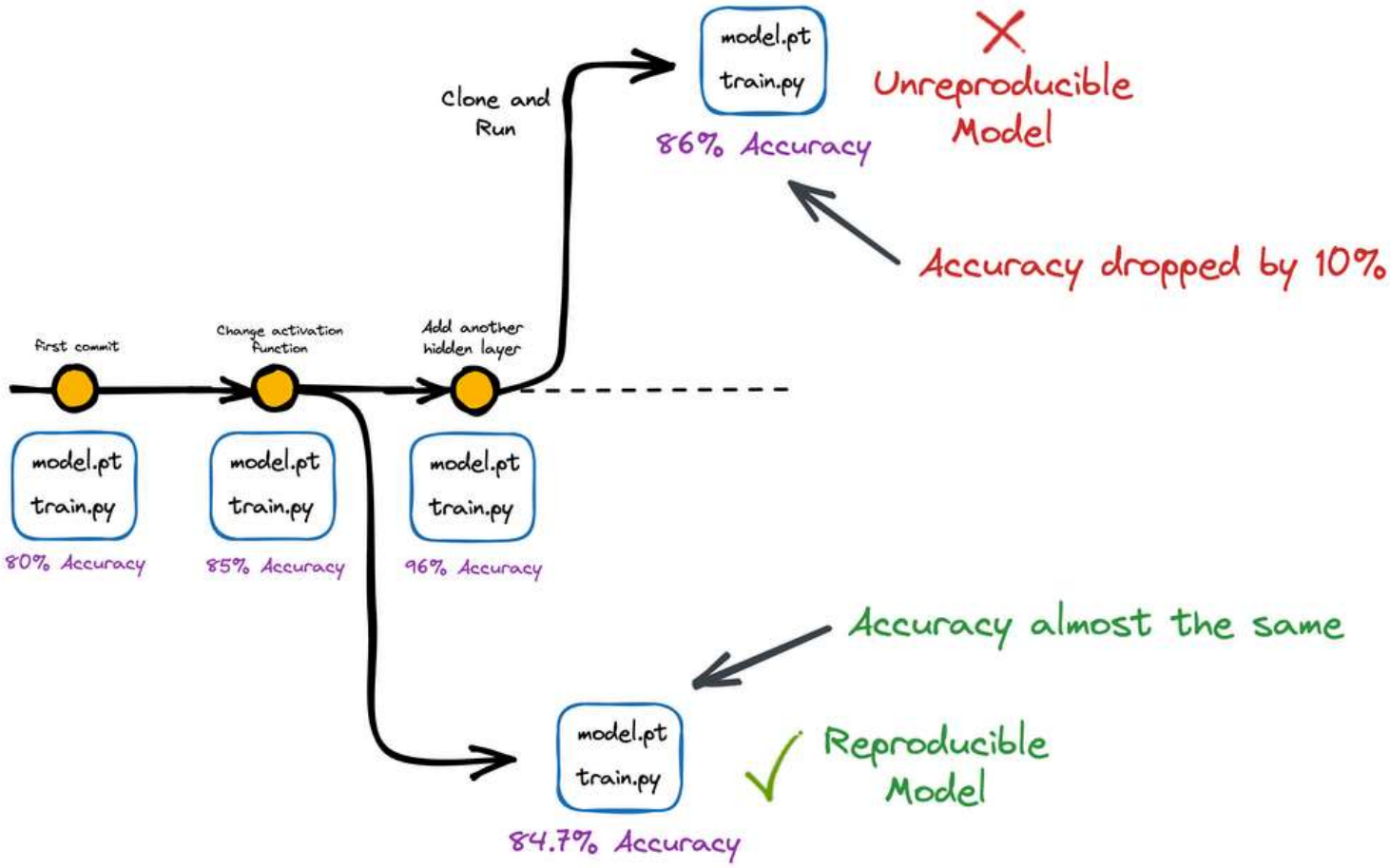



And it is well known that with version control, teams can work on the same codebase/data and improve the same models without interfering with each other's work.

Moreover, one can easily track changes, review each other's work, and resolve conflicts (if any).

**Lastly, version control also helps in the reproducibility of an experiment.**

It ensures that results can be replicated and validated by others, which improves the overall credibility of our work.




Version control allows us to track the exact code version and configurations used to produce a particular result, making it easier to reproduce results in the future.

This becomes especially useful for open-source data projects that many programmers may use.

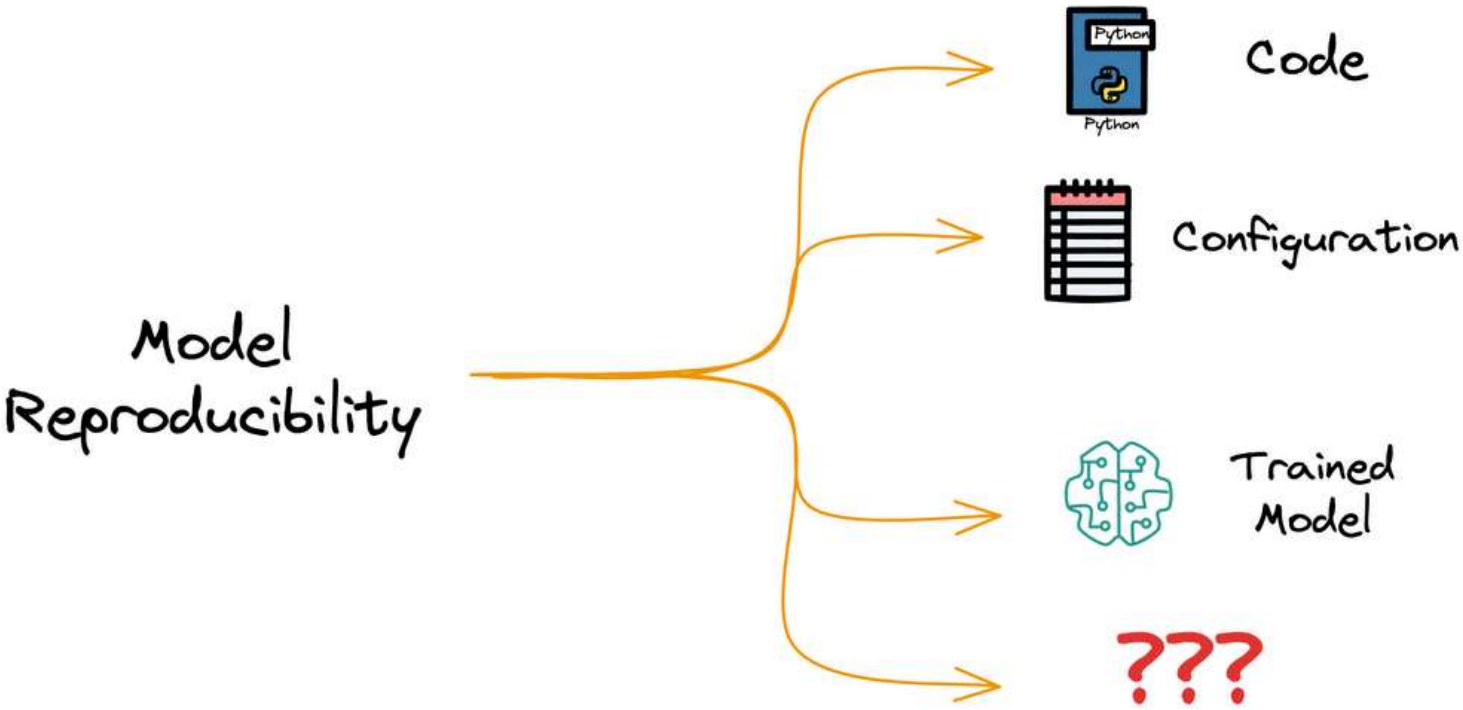
**HOWEVER!**

# Pillars of model reproducibility

Let me ask you something:

> *Purely from a reproducibility perspective, do you think model and code versioning are sufficient?*

In other words, are these the only requirements to ensure model reproducibility?




See, when we want to reproduce a model:

- First, we need the exact version of the code which was used to train the model.


    - We have access to the code through code versioning.
- Next, we need the exact configuration the model was trained with:


  - This may include the random seed used in the model, the learning rate, the optimizer, etc.
  - Typically, configurations are a part of the code, so we know the configuration as well through code versioning.
- Finally, we need the trained model to compare its performance with the reproduced model.


  - We have access to the model as well through model versioning.

But how would we train the model without having access to the **exact dataset** that was originally used?


In fact, across model updates, our data can largely vary as well.

Thus, it becomes important to track the exact version of the dataset that was used in a specific stage of model development and deployment.

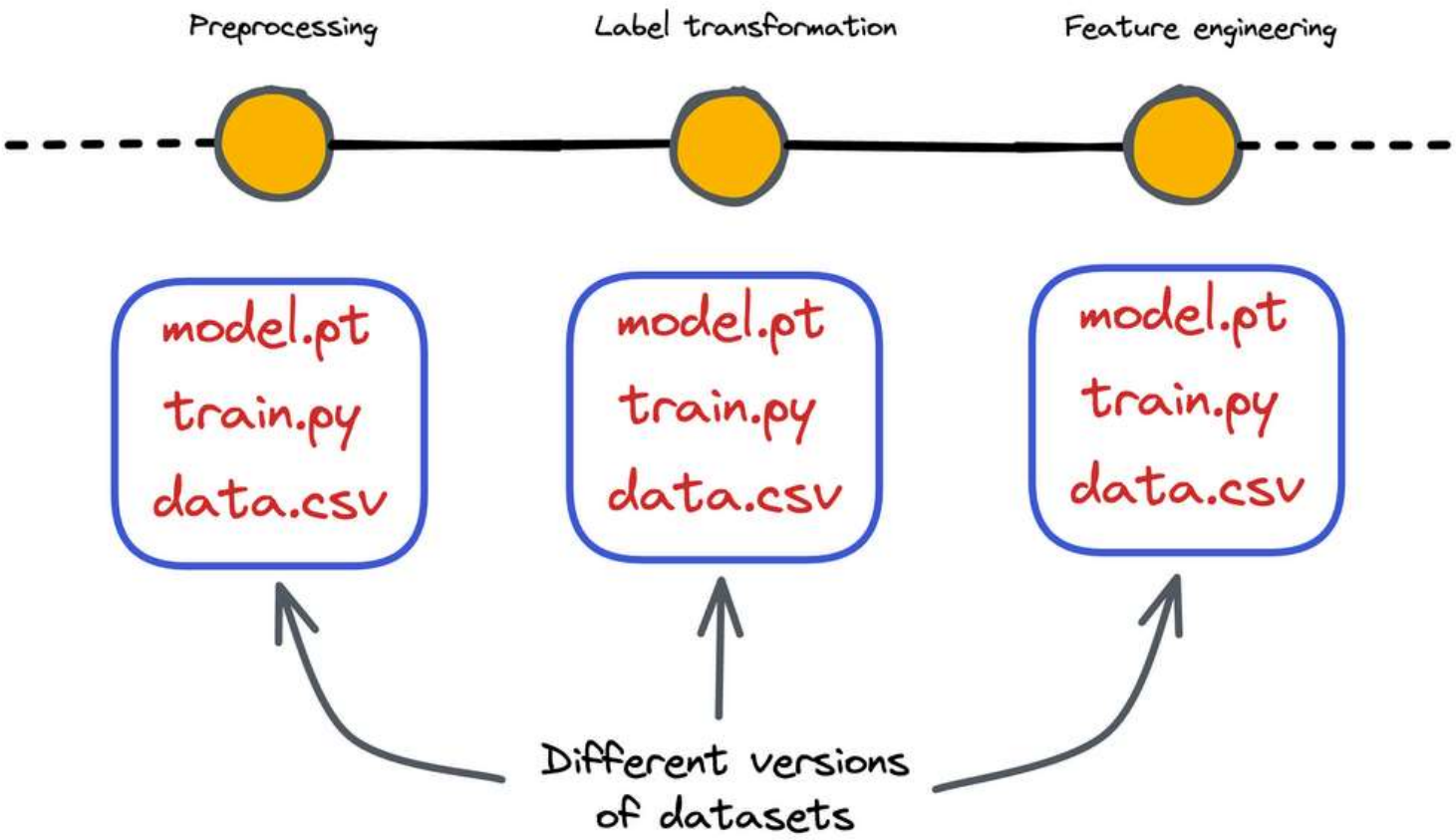
This is precisely what **data version control** is all about.

# Motivation for data version control

The motivation for maintaining a data version control system in place is quite intuitive and straightforward.

Typically, all real-world machine learning models are trained using large datasets.

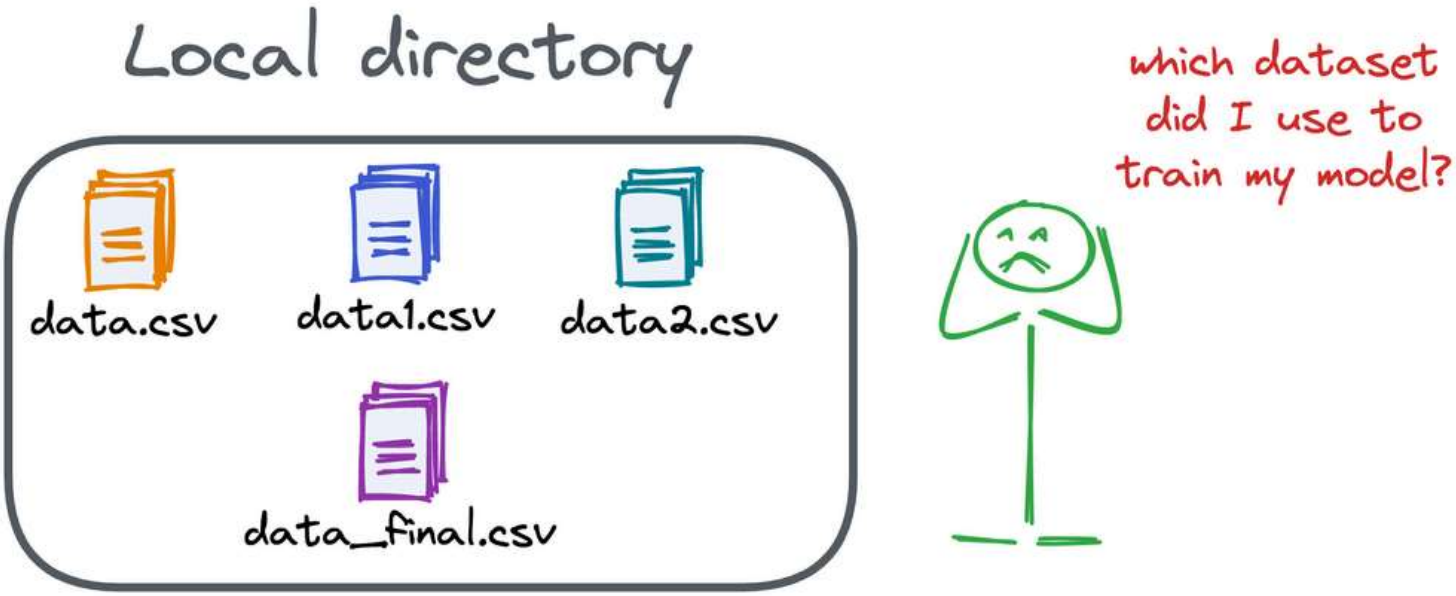
Before training a model and even during its development, many transformations are regularly applied to the dataset.




This may include:

- Preprocessing
- Transformation
- Feature engineering
- Tokenization, and many more.

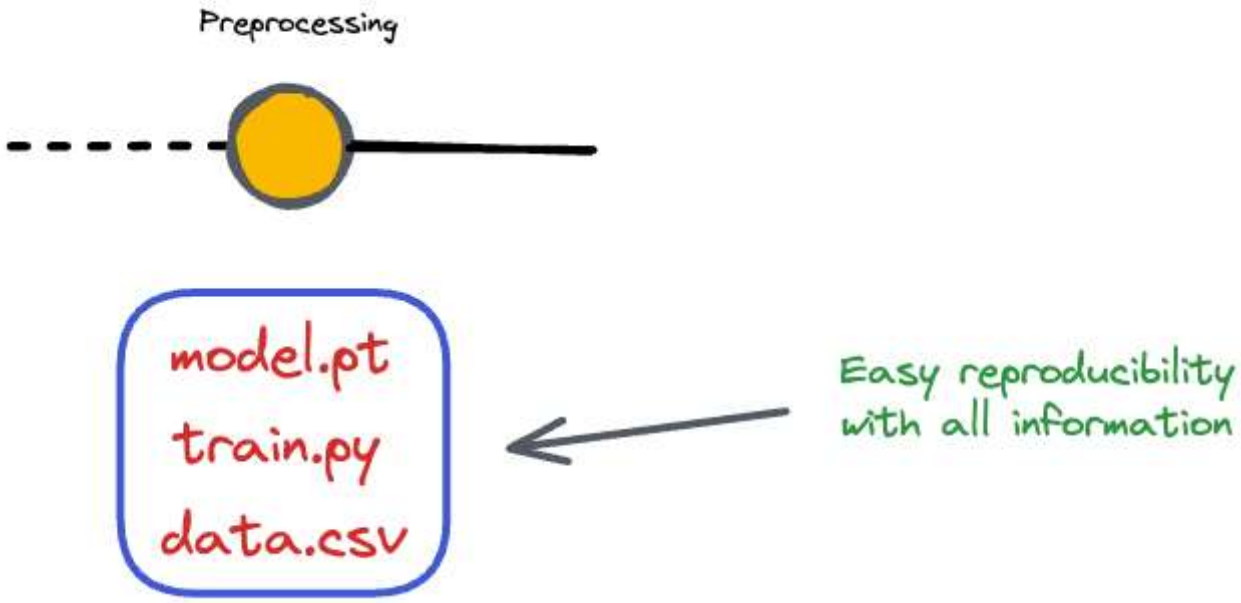
As the number of model updates (or iterations) increases, it can get quite difficult to track which specific version of the dataset was used to train the machine learning model.




If that is clear, then the motivation for having a data version control system in place becomes quite intuitive and simple, as it addresses many data-specific challenges:

## #1) Reproducibility

With a data version control system, we can precisely reproduce the same training dataset for any given version of our machine learning model.

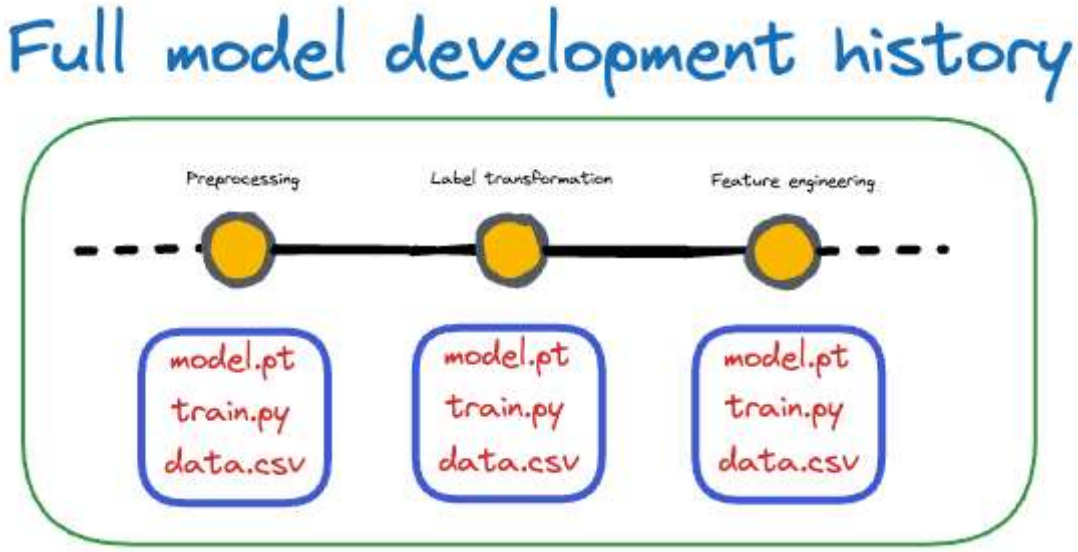



By tracking and linking each dataset version to a specific model version, we can easily recreate the exact conditions under which a model was trained, making it possible to replicate results.

## #2) Traceability

Like codebase traceability, data version control enables traceability by documenting the history of dataset changes.

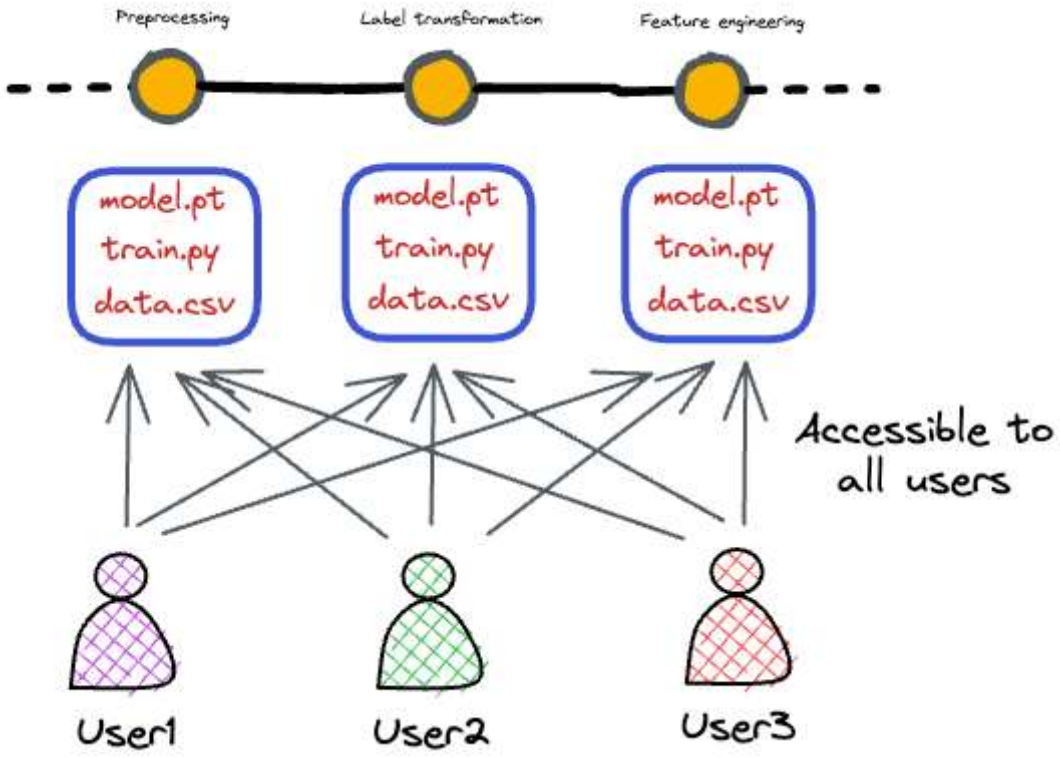
It offers a clear lineage of how the dataset has evolved over time, including information about who made changes, when those changes occurred, and the reasons behind those modifications.




This traceability is crucial for understanding the data's quality and history, ensuring transparency and accountability in our ML pipeline.

## #3) Collaboration and data sharing

In collaborative machine learning projects, team members need to work on the same data, apply transformations, and access shared dataset versions.



Data version control simplifies data sharing and collaboration by providing a centralized repository for datasets.

Team members can easily access and sync with the latest data versions, ensuring that everyone is on the same page.

## #4) Efficiency

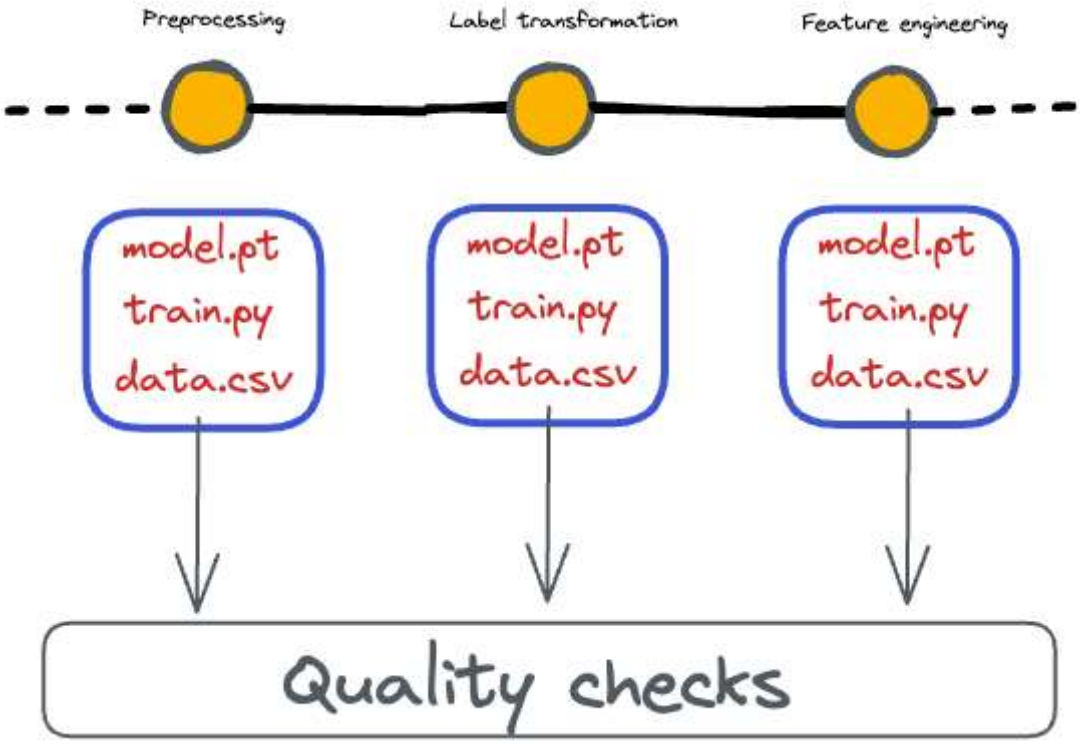
As we would see ahead in the practical demo, data version control systems help in optimizing storage and bandwidth usage by employing techniques such as:

- data deduplication
- data caching

This ensures that we do not waste resources on storing redundant copies of large datasets, making the storage and transfer of data more efficient.

## #5) Data quality control

It's common for datasets to undergo quality checks and transformations during their preparation.




Data version control can help identify issues or discrepancies in the dataset as it evolves over time.

By comparing different dataset versions, we can spot unexpected changes and revert to previous versions if necessary.

## #6) Data governance and compliance

Many industries and organizations have stringent data governance and compliance requirements.

Data version control supports these needs by providing a well-documented history of data changes, which can be crucial for audits and regulatory compliance.

## #7) Easy rollback

In situations where a newer model version performs worse or encounters unexpected issues in production, having access to previous dataset versions allows for easy rollback to a more reliable dataset.

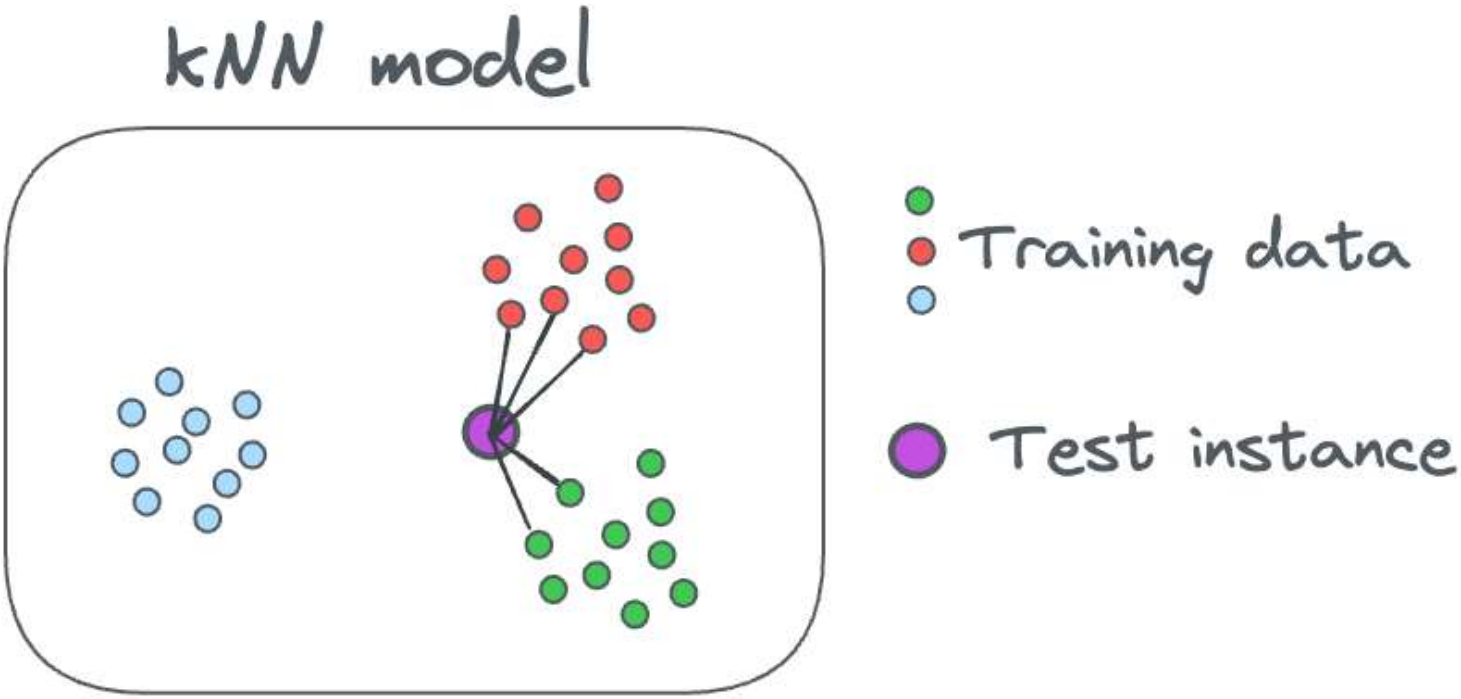
This can be a lifesaver in many situations.

**Here, you might be wondering, why data rollback might be of any relevance to us when it's the model that must be rolled back.**

You are right.

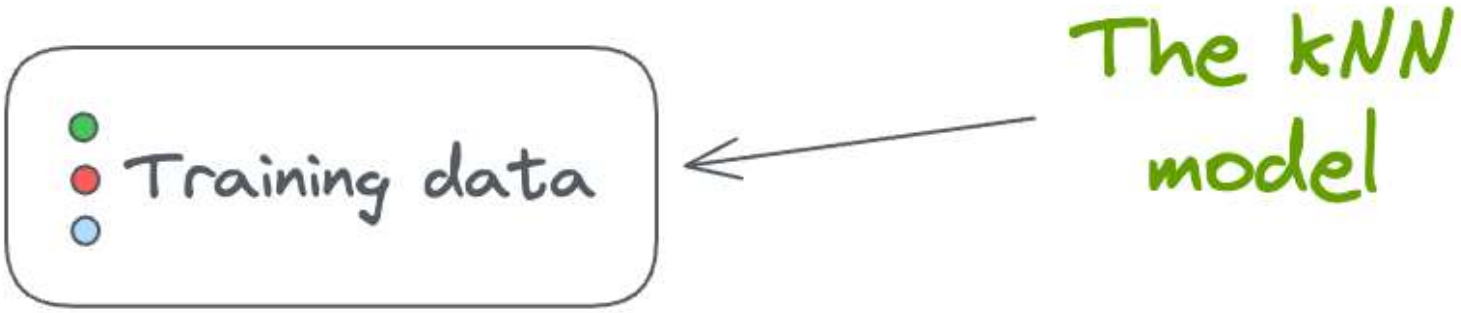
But there are situations where data rollback is useful.

For instance, assume that your deployed machine learning model is a k-nearest neighbor (kNN) model.




**The thing is that we never train a kNN.**

In fact, there are no explicitly trained weights in the case of a kNN. Instead, there's only the training dataset that is used for inference purposes.




The model is effectively the training data, and predictions are made based on the similarity of new data points to the stored instances.

If anything goes wrong with this dataset during production, having a data version control system in place can help us quickly roll back to a previous reliable dataset.

What could go wrong, you may wonder?

- Maybe the data engineering team has stopped collecting a specific feature due to a compliance issue.
- Maybe there were some human errors, like accidental data deletions. This will make the kNN void.
- Maybe there was data corruption — Data files or records became corrupted due to a hardware failure, network issues, or software bugs. This will make the kNN void.

The benefit of data version control is that if anything goes wrong with this dataset during production, having a data version control system in place can help us quickly roll back to a previous reliable dataset.

In the meantime, we can work in the development environment to investigate the issue and decide on the next steps.

# Considerations for Data Version Control

Now that we have understood the motivation for having a data version control system, let's look at some considerations for a **data version control** system.

# Why is Git not an ideal solution for data version control?

We know that Git can manage the versioning of any type of file in a Git repository. These can be code, models, datasets, config files, etc.

These can be hosted on remote repositories like GitHub with a few commands.

So for someone using GitHub to host codebases, they might be tempted to extend the same to version datasets:

- They may manage different versions of the dataset with Git locally.
- They may host the repository on GitHub (or other services) for collaboration and stuff.

The rationale behind this idea could be that Git can do data version control as elegantly as version control codebases.

Sounds like a fair thing to do, right?

Well, it's not!

Recall our objective again.

In the previous section, we discussed using data version control for production systems.

As production systems are much more complex and involve collaboration, the codebase is typically hosted in a remote repository, like on GitHub.

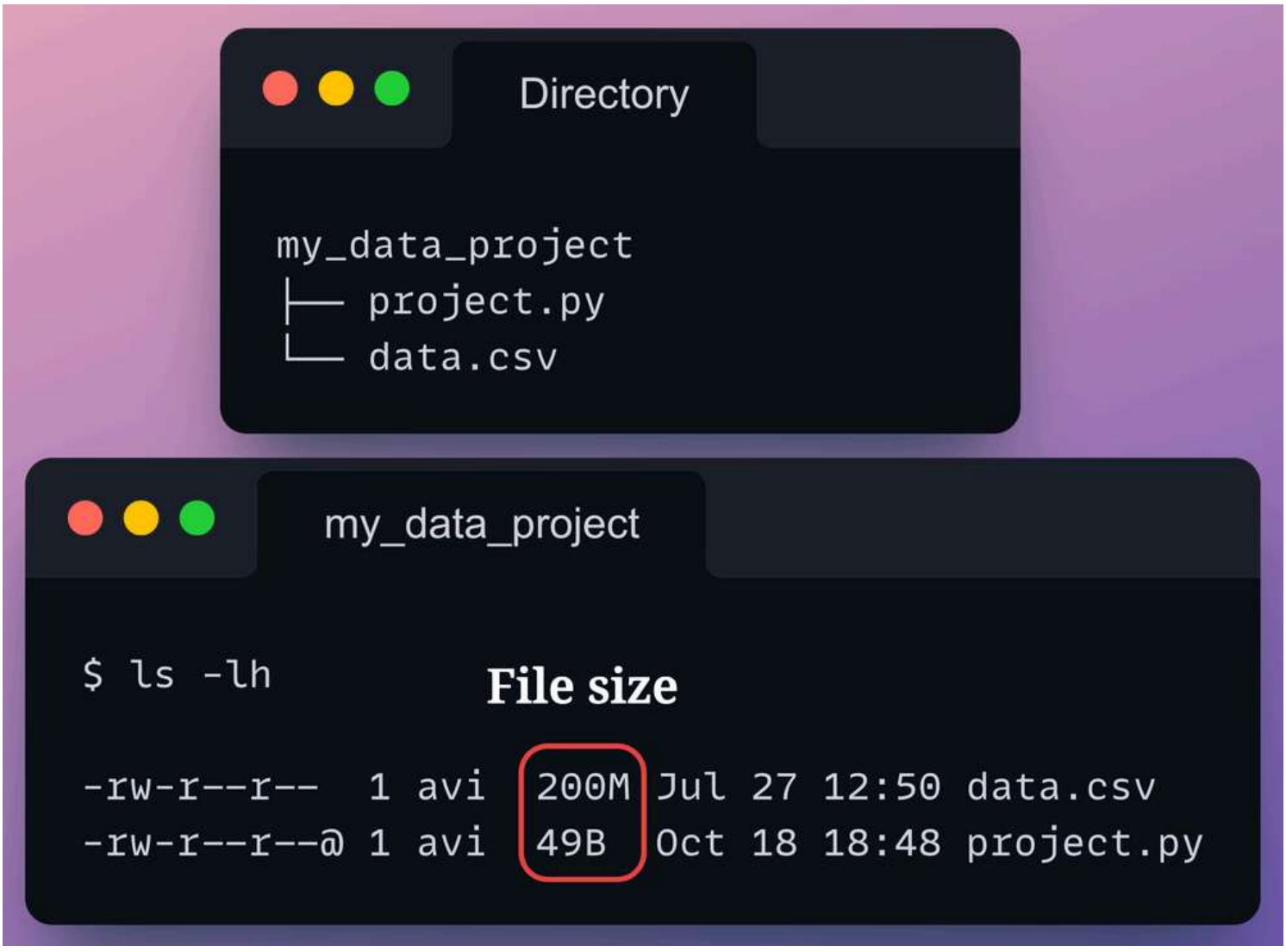
But the problem is that Github repositories (or other similar hosting tools like GitLab) always have an upper limit on the file size we can push to these remote repositories.

In other words, GitHub is only designed for lightweight code scripts. However, it is not particularly well-suited for version controlling large datasets that exceed just a few MBs in size.

Typically, in machine learning projects, the dataset size can be in the order of GBs. Thus, it is impossible to execute data version control with Github.

In fact, we can also verify this experimentally.

Consider we have the following project directory:



As shown above, the `data.csv` file takes about 200 MBs of space.

Let's create a local Git repository first before pushing the files to a remote repository hosted on GitHub.

To create a local Git repository:

- First, we shall initialize a git repository with `git init`.
- Next, we will add the files to the staging area with `git add`.
- Finally, we will use `git commit` to commit to the local repo.

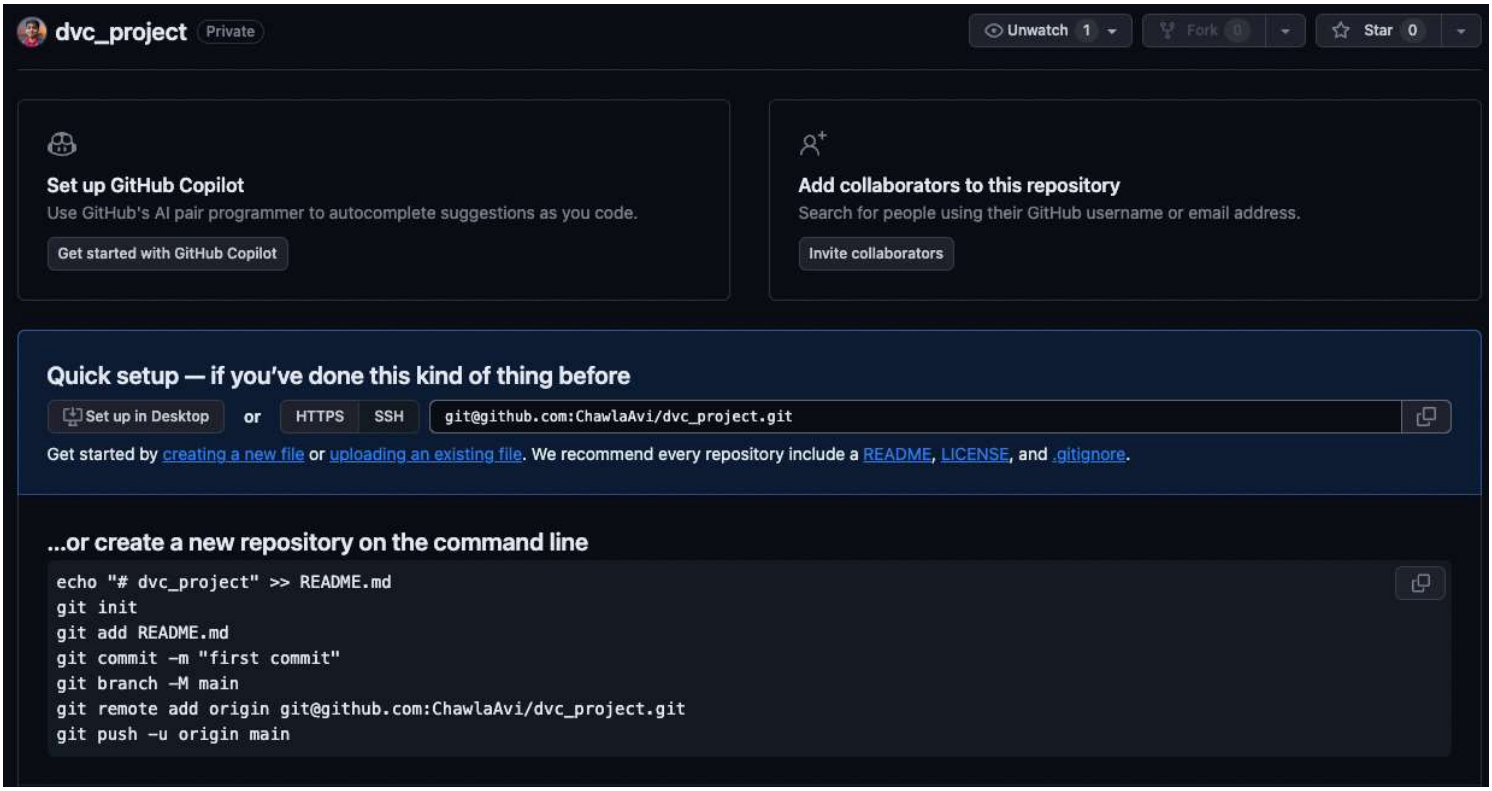
This is demonstrated below:

👉 This is not shown here but we will do the same for the `project.py` file as well.

All good so far. The changes have been committed successfully to the local git repo.

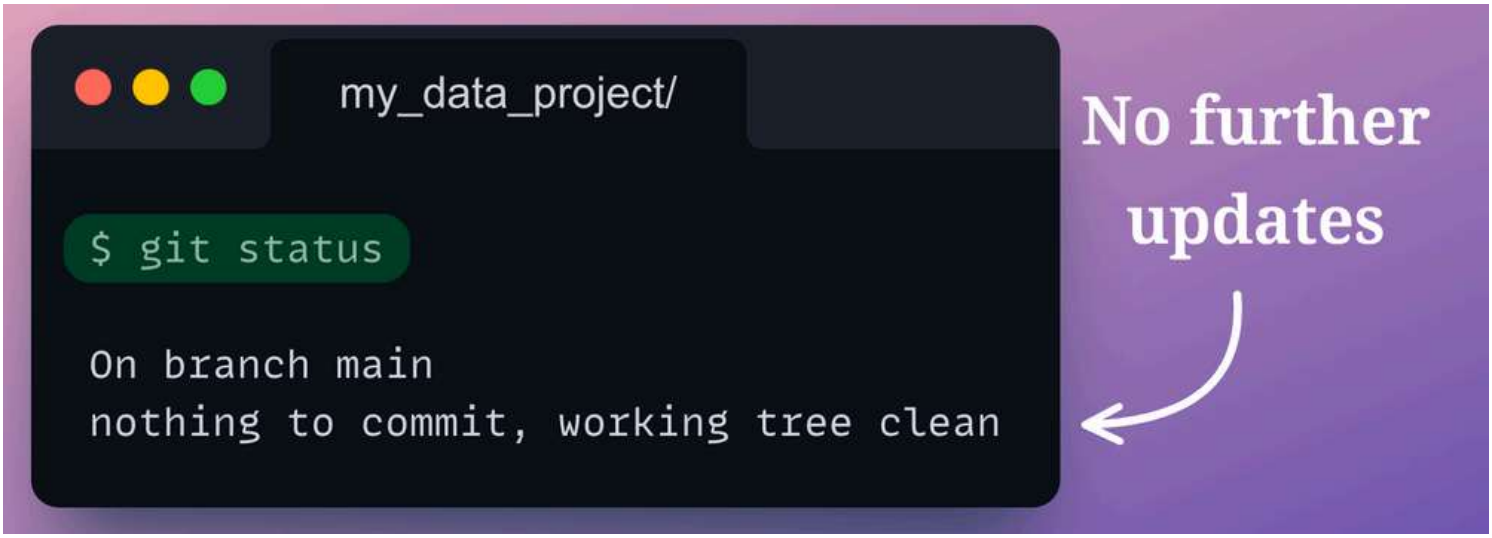
We can verify this by reviewing the commit history of the local git repo with `git log` command.


Next, let's try to push these changes to a remote GitHub repository. To do this, we have created a new repository on GitHub – `dvc_project`.




Before pushing the files to the remote repository, let's also verify whether the local git repository has any uncommitted changes using the `git status` command:

💡 `git status` command is used to check the status of your git repository. It shows the state of your working directory and helps you see all the files that are untracked by Git, staged, or unstaged,




The output says that the working tree is clean.

Thus, we can commit these changes to the remote GitHub repository created above as follows:

As discussed earlier, Github repositories always have an upper limit on the file size we can push to these remote repositories. This is precisely what the above error message says.

For this reason, we can only use the typical GitHub repositories for data version control as long as the dataset size is below a few MBs, which is rarely the case.

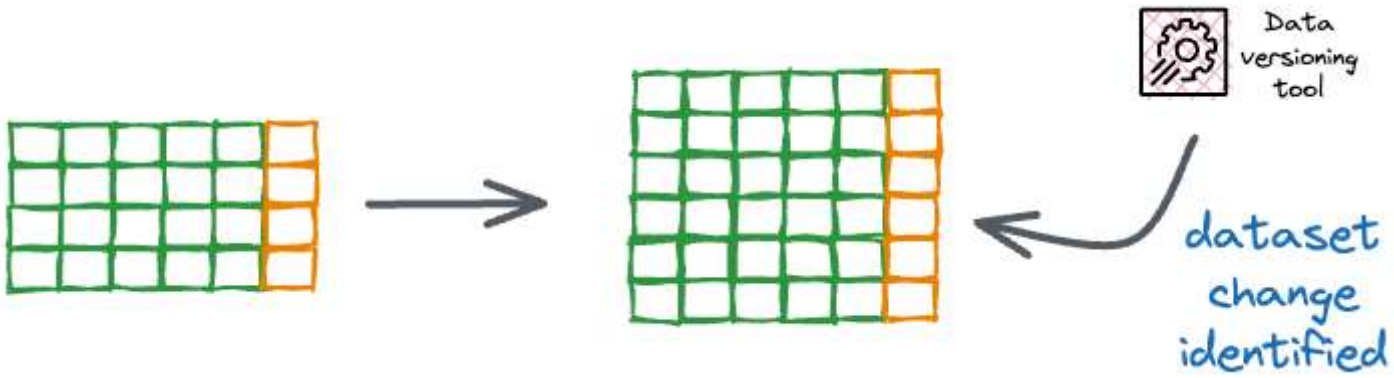
Thus, we need a better version control system, especially for large files, that does not have the above limitations.

> 💡 Before we proceed, a critical point to note here is that as long as we are **only working locally** and wish to maintain data version control in our personal machine learning projects, using the usual git-based functionalities is not a big problem. You can use Git if you want as long as you keep everything on our local computer. However, versioning large files is often time-consuming with Git so it is recommended to use tools that are specifically meant for such purposes.

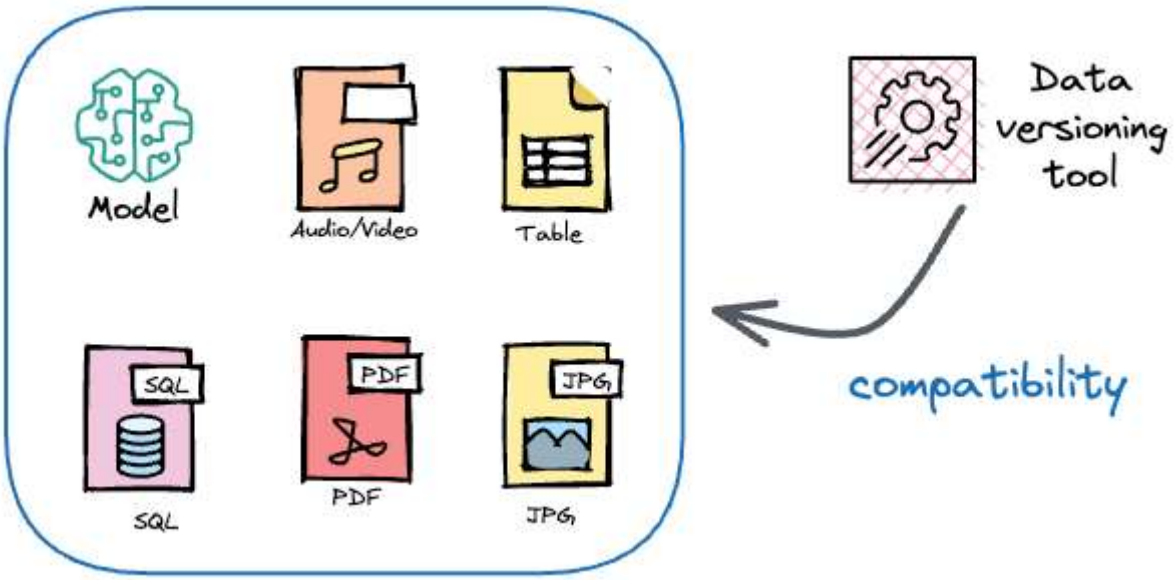
# Ideal requirements for a data version control system

An ideal data version control system must fulfill the following requirements:

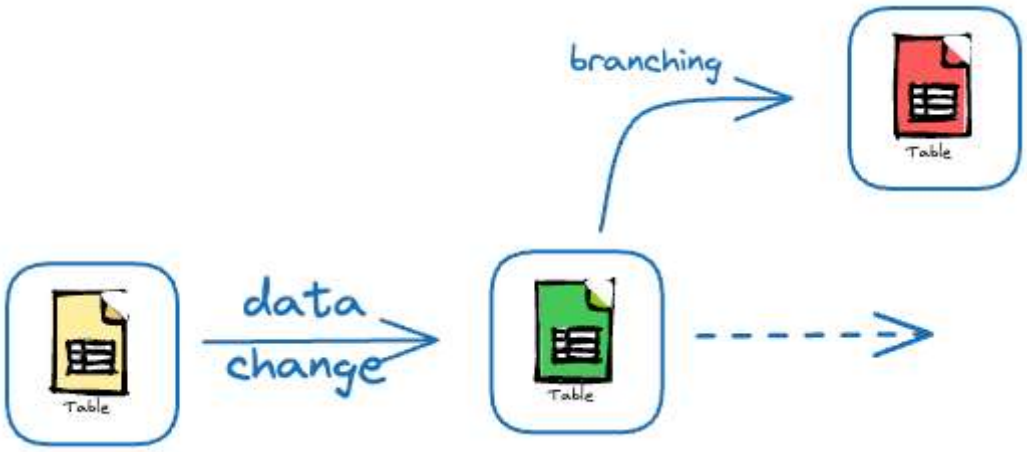
- **It should allow us to track all data changes like Git does with files.**
  - As soon as we make any change (adding, deleting, or altering files) in a git-initialized repository, git can always identify those changes.



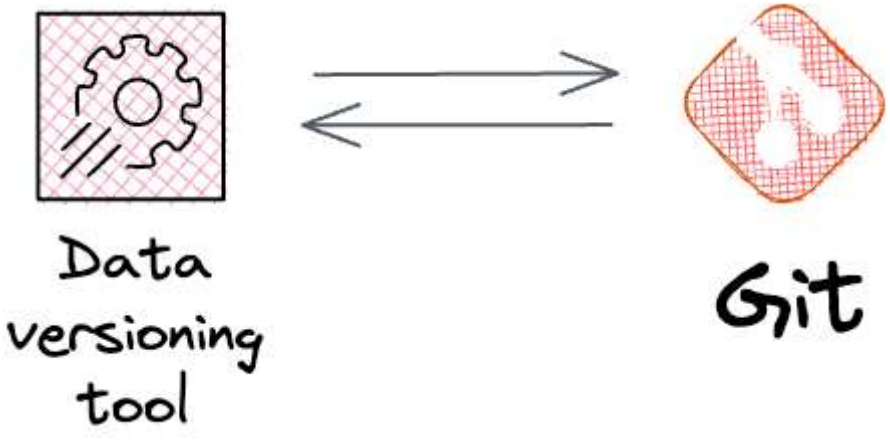

  - The same should be true for our data version control system.
- **It should not be limited to tracking datasets.**
  - See, as we discussed above, GitHub sets an upper limit on the file size we can push to these remote repositories.




  - But large files may not not necessarily be limited to datasets. In fact, models can also be large, and pushing model pickles to GitHub repositories can be difficult.
- **It must have support for branching and committing.**




    - Like Git, this data version control system must provide support for creating branches and commits.
- **Its syntax must be similar to Git.**
    - Of course, this is optional but good to have.
    - Having a data version control system that has a similar syntax to git can simplify its learning curve.
- **It must be compatible with Git.**
    - In any data-driven project, data, code, and model always work together.
    - After using the data version control system, it must not happen that we are tracking code with Git and then managing models/datasets with an entirely non-compatible tool.



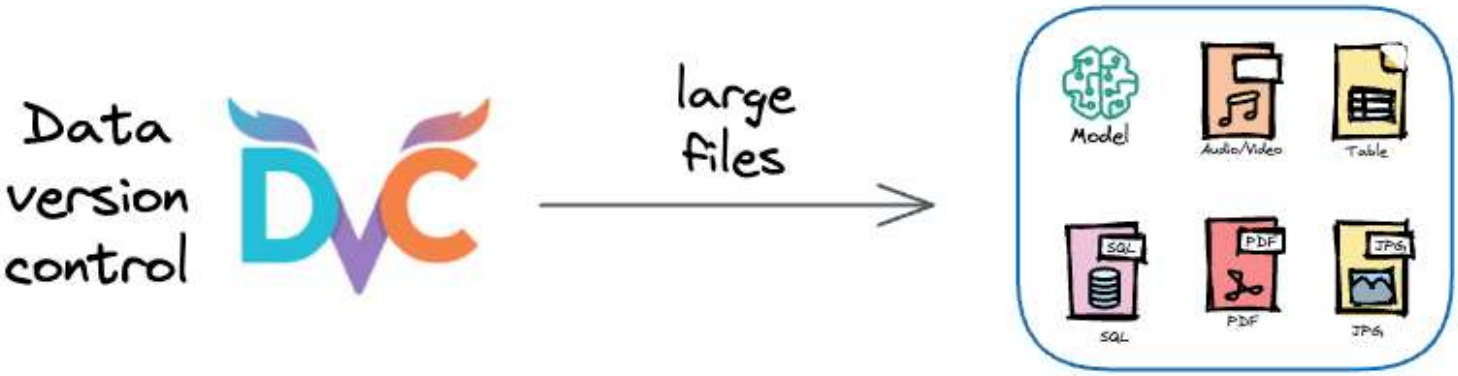

    - They must work seamlessly integrated to avoid any unnecessary friction.
- **It must have collaborative functionalities like Git**

- Like git makes it extremely simple for teams to collaborate and work together by sharing their code, the data version control must also promote collaboration.

So what can we do here?

Thankfully, similar to Git, which is primarily used for script versioning, there is a tool called **DVC**, which is used for data version control.


You can think of it like Git but specifically built for version-controlling large files.


When I say “large files,” it naturally means that DVC is not just limited to version-controlling datasets, but we can also version-control any large file like model

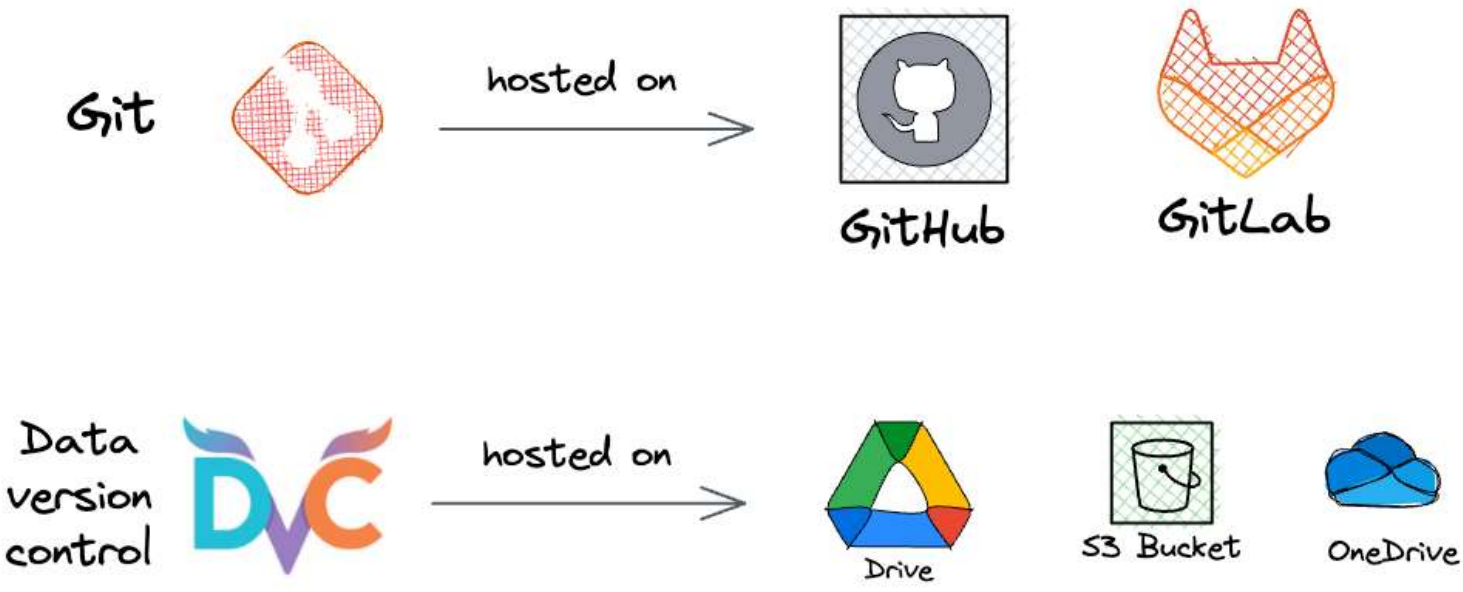
binaries, large zip files, and more.

> **DVC** is the name of the tool, and it stands for **data version control**. Going ahead, whenever I will mention "DVC," it would refer to the tool. But whenever I will mention "data version control," it would refer to the activity of versioning the dataset.

## How does DVC work?

We know that Git lets us store our codebase on remote hosting services like GitHub or GitLab.

Similarly, DVC lets us upload our dataset and models to remote storage, which are typically centralized.




Being centralized (like a GitHub repository), anyone in the team can access it. They can create branches, commit changes, etc.

This remote storage can be driven by any cloud service provider like AWS S3, Microsoft Azure Storgrae, or even your Google Drive.

And as mentioned above, this remote storage can act as a single source of storage for your whole project/team, just like a GitHub repository.

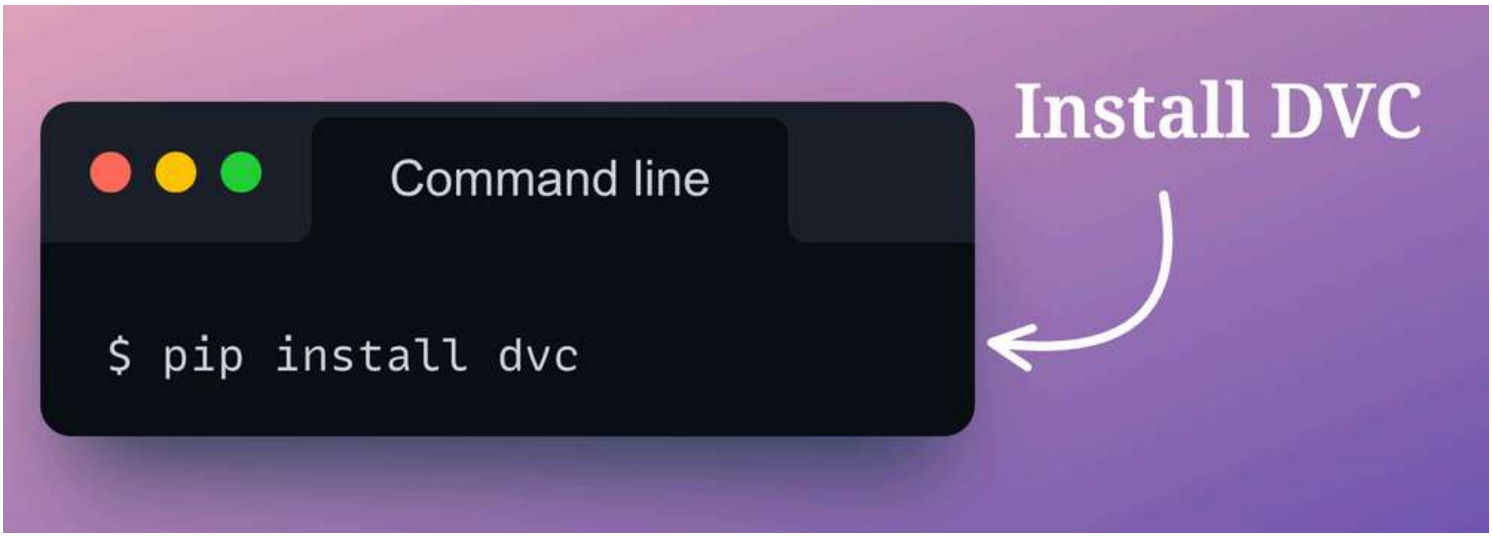
# Versioning Data with DVC

Before getting into much of the advanced features and details of DVC, let's cover some core features of DVC, such as:

- How to add datasets and models to DVC tracking.
- How to push (or pull) those datasets and models to (or from) the centralized cloud storage
- How to use git to roll back to previous versions of our datasets and models.

> *The entire demonstration will be beginner-friendly. For better understanding, I would recommend you to open a terminal and get started. I have also provided a practice exercise towards the end so if you want, you can read through the article first and then try that exercise to solidify your understanding.*

First, to use data version control functionalities from DVC, we must install the DVC tool. We can do this using `pip` as follows:




The directory structure is somewhat similar to what we discussed earlier:


As shown above, we have a large dataset that cannot be pushed to GitHub.

To begin, we first inialize the `dvc_guide` directory as a git repository and a DVC repository using `init` commands as follows:

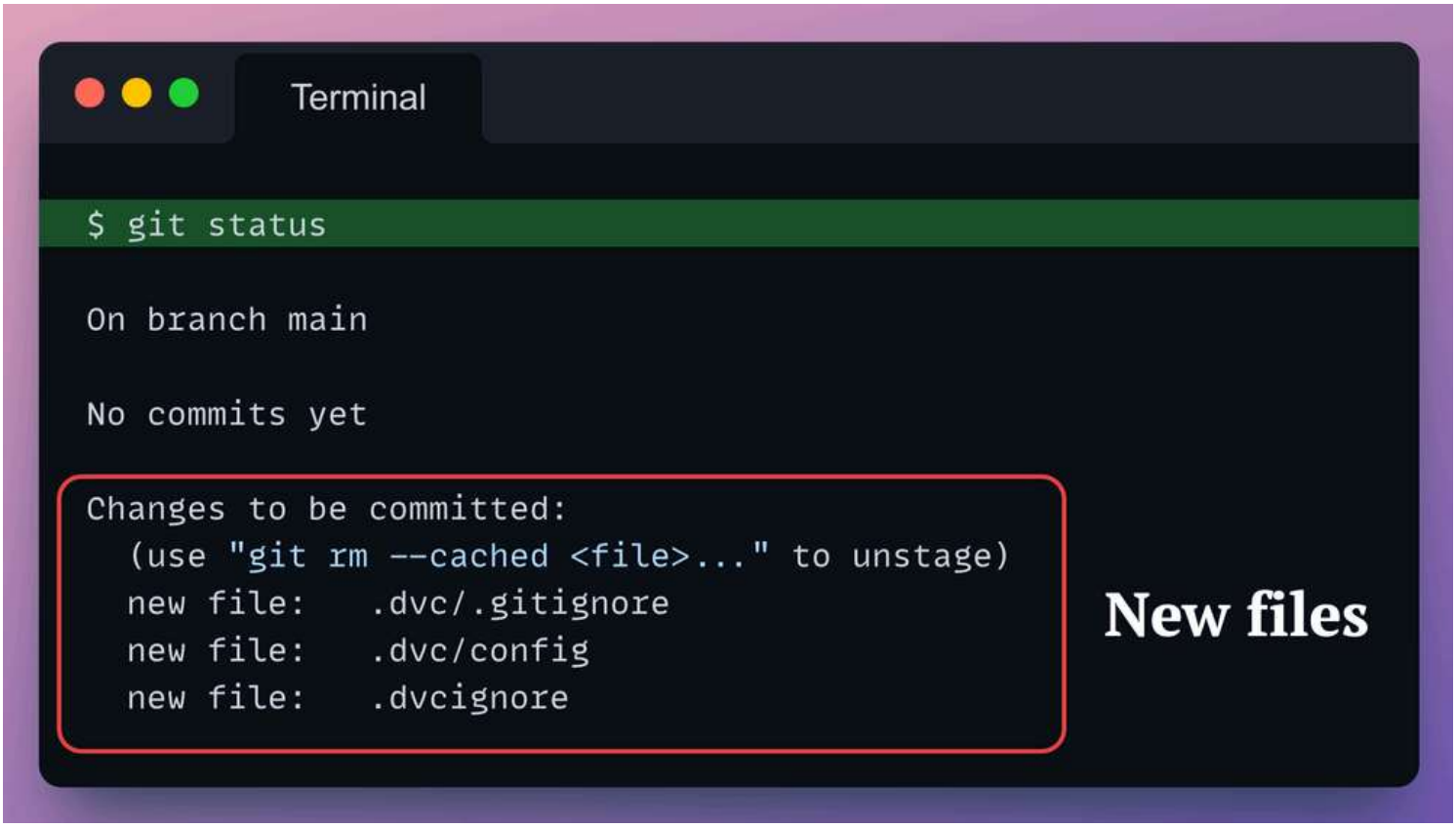

- We use `git init` to initialize a git repository.
- Next, we use `dvc init` to initialize a dvc repository.

Done!

Running the `dvc init` command adds a few files to the current repository.

We can see them if we run the `git status` command.

> 💡 `git status` command is used to check the status of your git repository. It shows the state of your working directory and helps you see all the files which are untracked by Git, staged, or unstaged,



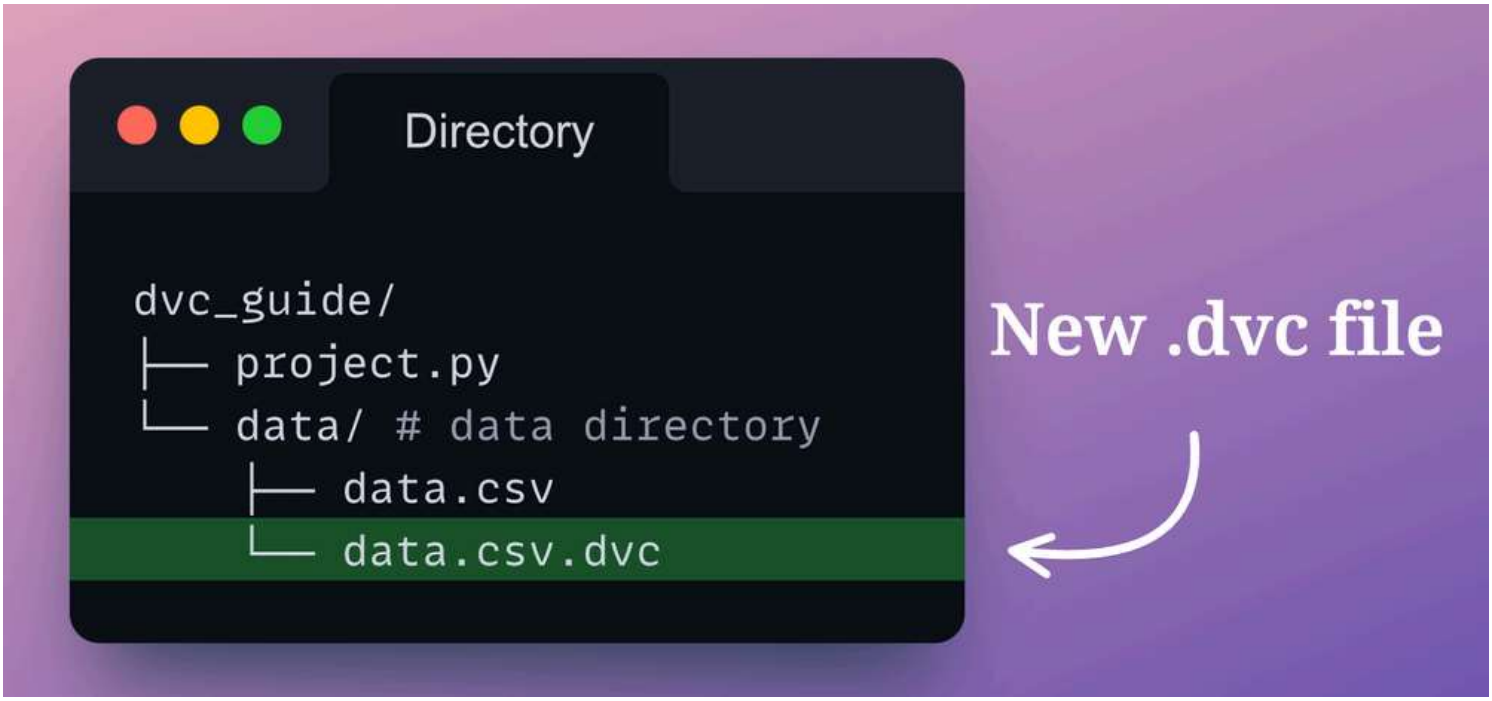

We'll understand the purpose of these files shortly, but for now, let's commit them to the local Git repository:

As expected, when we run `git status`, we see that there are a few more files that are yet to be staged and committed to the local repository.

Next, let's add our dataset to DVC tracking.

When using Git, we add files to Git tracking using `git add`. Similarly, to add files to DVC tracking, we run `dvc add`. This is demonstrated below:

After running the `dvc add` command, look at the directory structure:




*Ignore all hidden files (those that start with '.')*

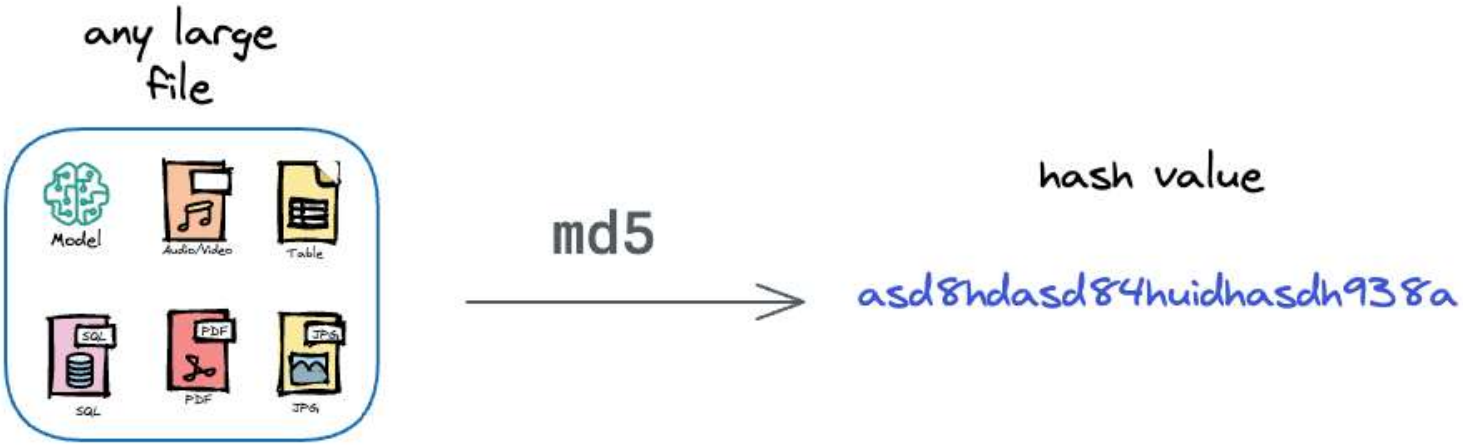
Running the command adds a new `data.csv.dvc` file. Also, after running `dvc add`, DVC asked us to run another command: `git add data/.gitignore data/data.csv.dvc`. We'll get back to it shortly.

First, let's look at the size and the contents of this `data.csv.dvc` file:

- The `data.csv.dvc` file is small, only `93B`.
- The contents of the file include two types of things:
  - A hash value of the file that was added with DVC.
  - The metadata of the file, such as its size, the type of hash used (`md5` in this case) and the path to that file.

The hash value is extremely useful to identify the dataset version.

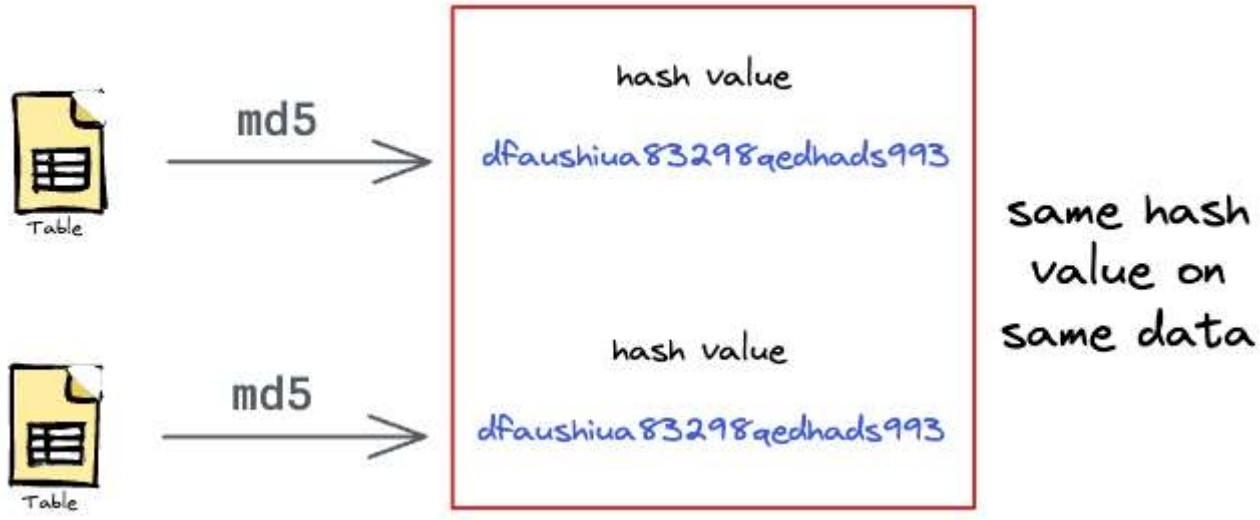
Essentially, the hash function (`md5` in this case) takes a file of arbitrary size to produce a string of length `32` characters.




This set of characters can appear a bit mysterious and random, but this string is meaningfully linked to our data.

**The thing is that the hash value will always be the same if we rerun the command on the same file repeatedly.**

In other words, the hash value will not change as long as the contents of the file remain unchanged.




But even if a single bit gets changed, the entire hash value will change.

Also, the hash value is always unique for a specific dataset — there are never any collisions.

As we saw above, the `data.csv.dvc` file holds a unique identifier for our dataset.

**Being unique, the core idea behind data version control using DVC is to version this `data.csv.dvc` file in git instead of the entire dataset.**

This is practically feasible because as we saw earlier, the `data.csv.dvc` file generated with the `dvc add` command is lightweight.


Therefore, instead of versioning the dataset, this `data.csv.dvc` file can be easily versioned with Git pushed to GitHub.

Simultaneously, DVC (the tool) can help us establish a connection to the data hosting service, seamlessly upload datasets, and associate each uploaded dataset with its `md5` hash value.

This will work because the hash function is always unique. In other words, there can be any collisions in hash value across multiple datasets.

By versioning the `data.csv.dvc` file in Git, we can track the precise changes in the hash value.

During a specific commit, say we want to pull the dataset used.

It's easy because DVC can look up the hash value present in the `data.csv.dvc` file of the current commit and pull the specific dataset from the data hosting service whose hash value matches with the hash value in the `data.csv.dvc` file.

Pretty neat and clever, right?

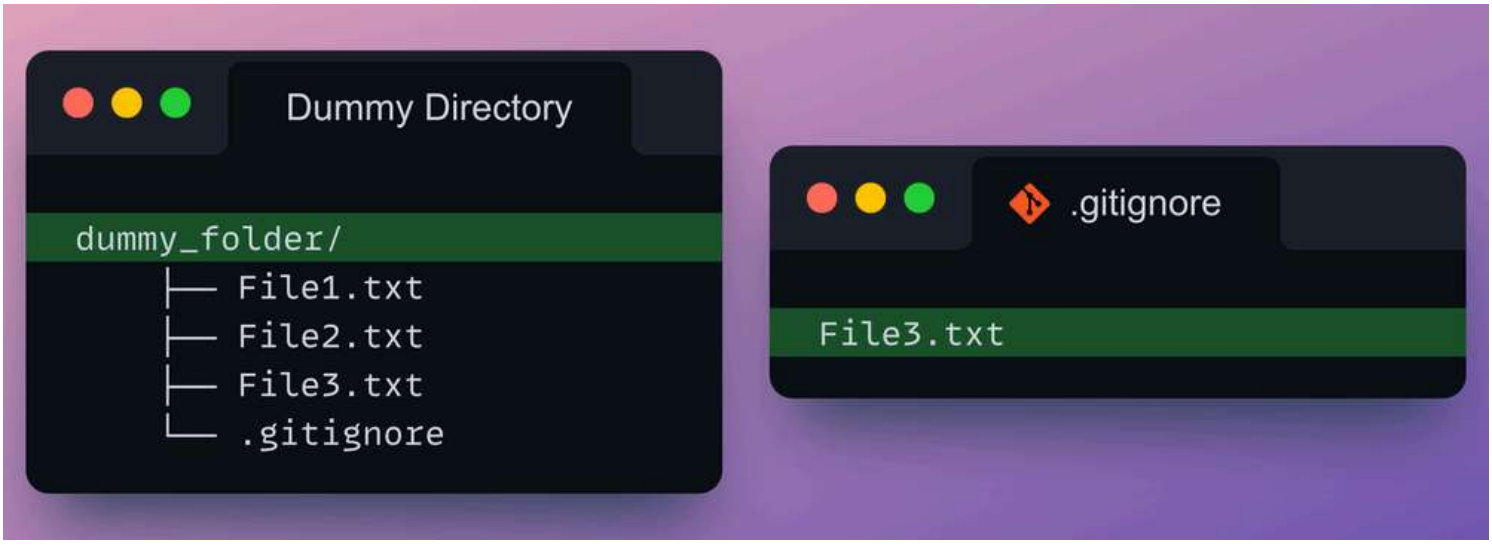
That is why the `dvc add` command we ran earlier instructed us to add a couple of new files to Git:

The output command says:


First, it asks us to add the `.gitignore` file for git tracking.

If you don't know what `.gitignore` does, no worries. As the name suggests, the `.gitignore` file is used to specify the files that we don't want Git to version.

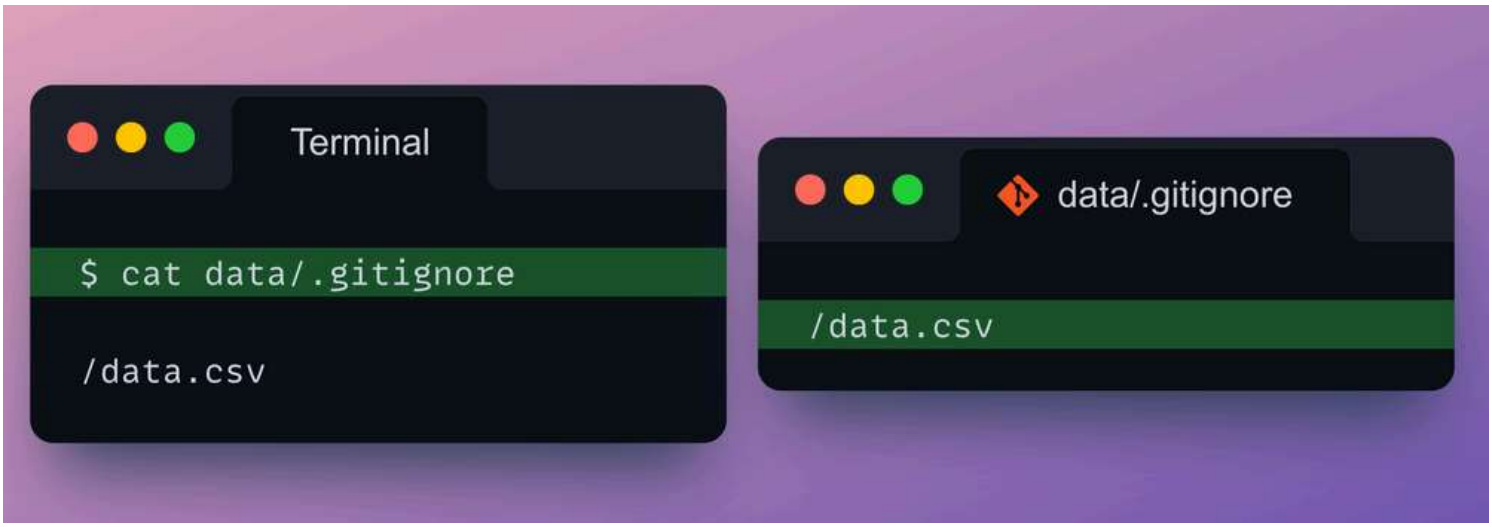
For instance, say our directory has the following files and the `.gitignore` file is as follows:




This would mean that during git tracking, any changes made to `File3.txt` will be entirely ignored by Git.

Based on this idea, we would expect the `data/.gitignore` file to list the `data.csv` file because we don't want Git to track the file.

Let's see if that is true:



We are correct. The `.gitignore` file lists the `data.csv` file – asking Git to ignore any updates to this file.

During DVC add, DVC automatically added this `.gitignore` file.

At the same time, DVC added the `data/data.csv.dvc` file, which holds the hash value and other metadata.

In the output of the `dvc add` command, DVC did ask us to version this `data.csv.dvc` file:


This sounds pretty logical and intuitive as well.

Simply put, instead of tracking the entire dataset, DVC asks Git to track another file (the `.dvc` file) that holds a unique identifier (hash value) for that dataset.

Let's run what DVC has asked us to do.

First, let's stage and commit the `.gitignore` file:

Next, let's do the same for the `data.csv.dvc` file:

So, at this point, we have got a dataset that is tracked by DVC, and we have its unique hash file, which is being tracked by Git.

However, the dataset is available on our local machine, and we want to push it to remote storage.

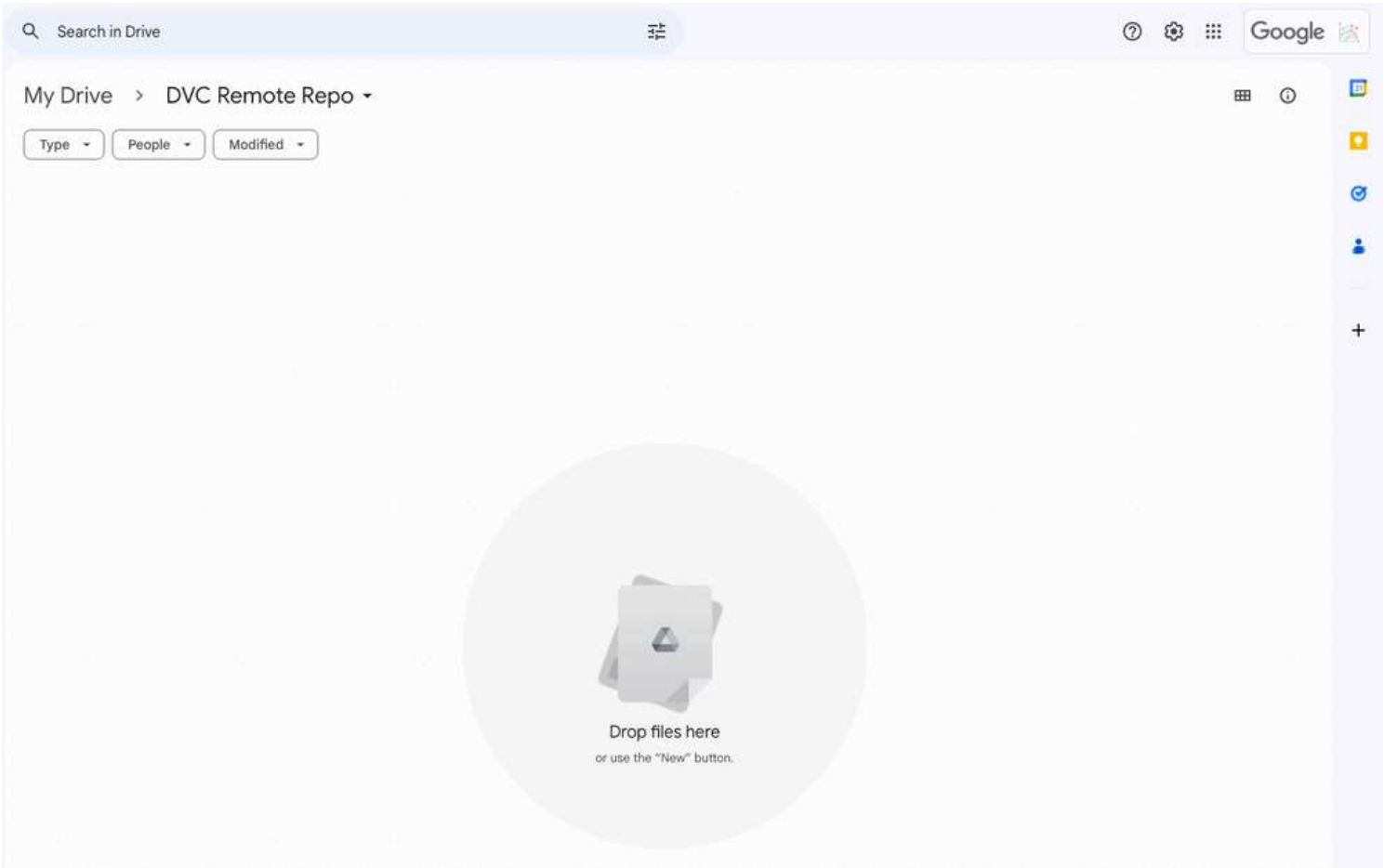
When we want to add a remote repo to a Git repository, we add a `git remote add`.

**Similarly, if we want to add a remote storage for DVC, we will run ...? Can you guess? Click the down arrow to know the answer.**

In this demonstration, let's do this with Google Drive as almost everyone has access to it, and it's fairly simple for anyone to understand how it works.

> *That said, with DVC, you can translate the same methodolody to many another remote storage of your choice.*

To begin, first, open **Google Drive** and create a new folder:




*Empty repository in Google Drive*

From the URL of the folder, grab the ID after the substring `folders/`:


Next, we will let DVC connect to this remote storage to upload datasets

Yet again, the syntax is similar to the command we use when we wish to add a GitHub repository as a remote repository:

```
git remote add origin git@github.com:...
```

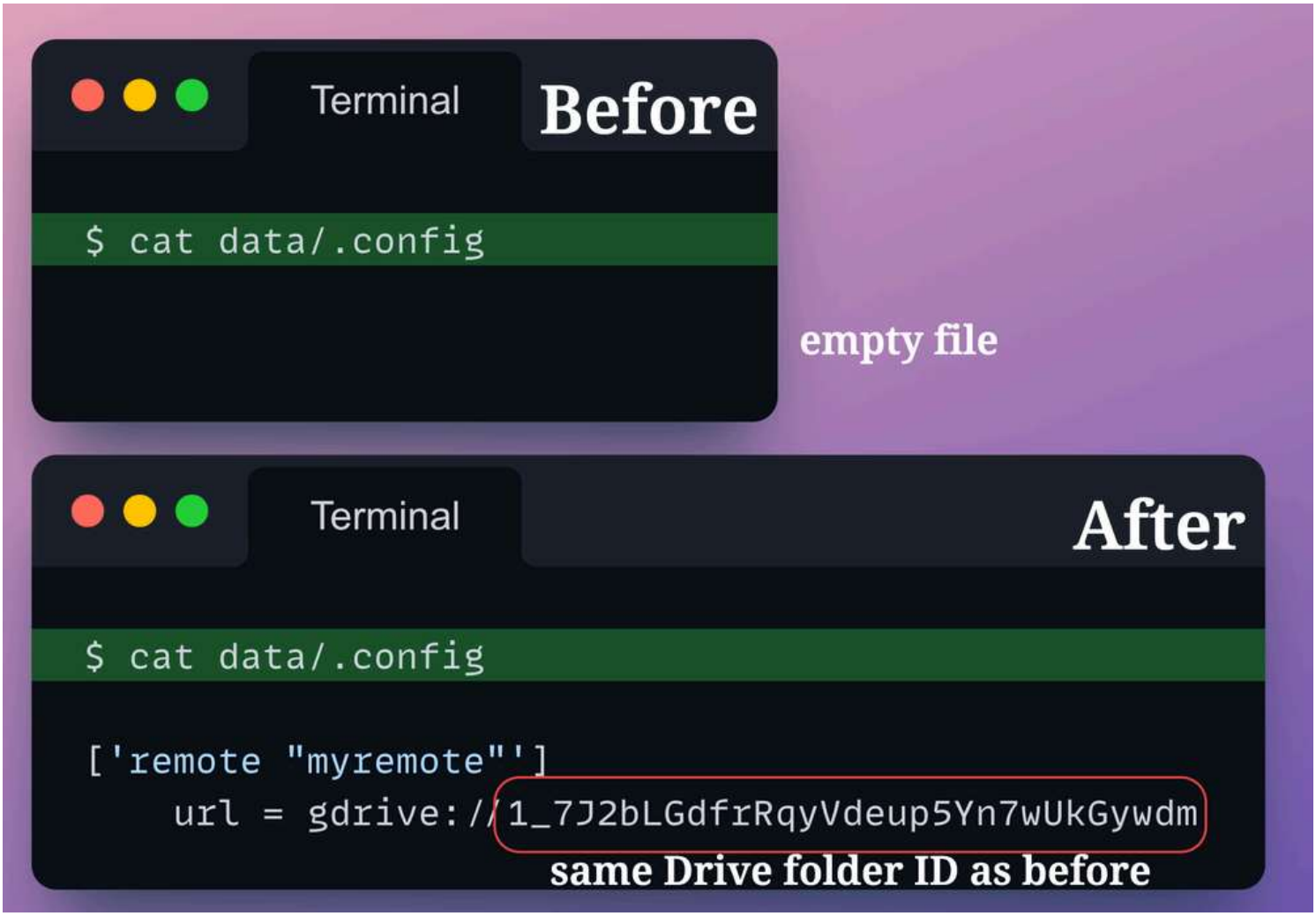
Copy

Similarly, to add a remote storage, we run the following:

Simply put, this is the command to add the Google Drive folder as a remote storage for DVC. In the above command, `myremote` indicates a human-readable name/alias for the cloud storage.

At this point, if we do `git status`, we will notice that Git has identified some changes to the local repository:


Let's check the contents of `.dvc/config`:




Interesting. After configuring the remote storage, the `.dvc/config` file lists the location of remote storage.

Now you know the purpose of `.dvc/config` file.

**This file tells DVC the locations of remote storage it can push datasets to and download versioned datasets from when needed.**

> 💡 Of course, in this case, we have added only one remote location. But you can add multiple remote storage locations with `dvc remote add`. That is why the alias is useful. It helps us specify which location we want to push datasets to and where to pull them from if needed.

So, coming back, we noticed above that after running the `dvc remote add` command, the `.dvc/config` got modified.

Let's commit that to the local repository before proceeding ahead.

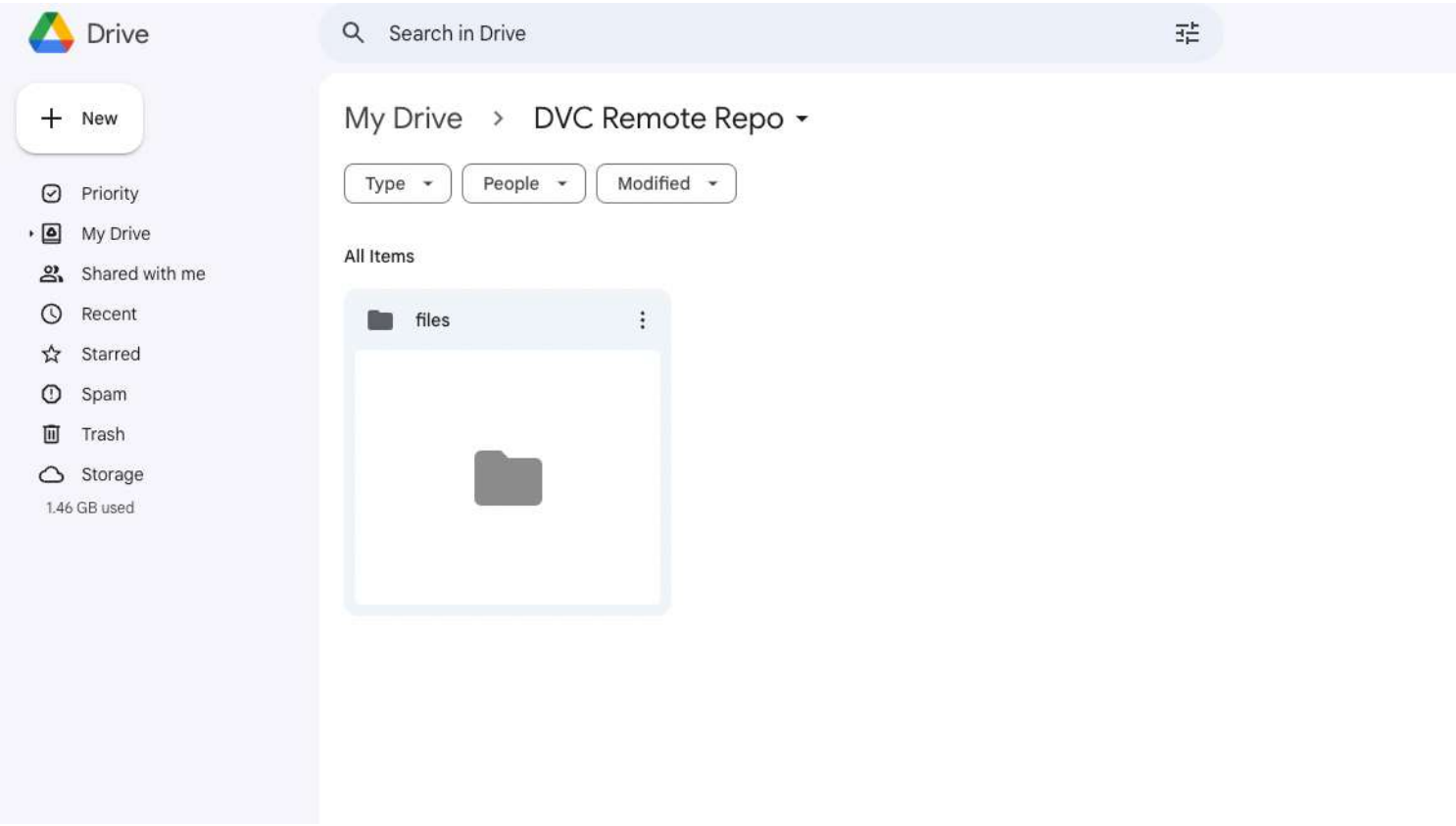
Now, we are ready to push the dataset to remote storage.

When pushing to GitHub, we run `git push`. Similarly, in this case, instead of using `git`, we use `dvc`.

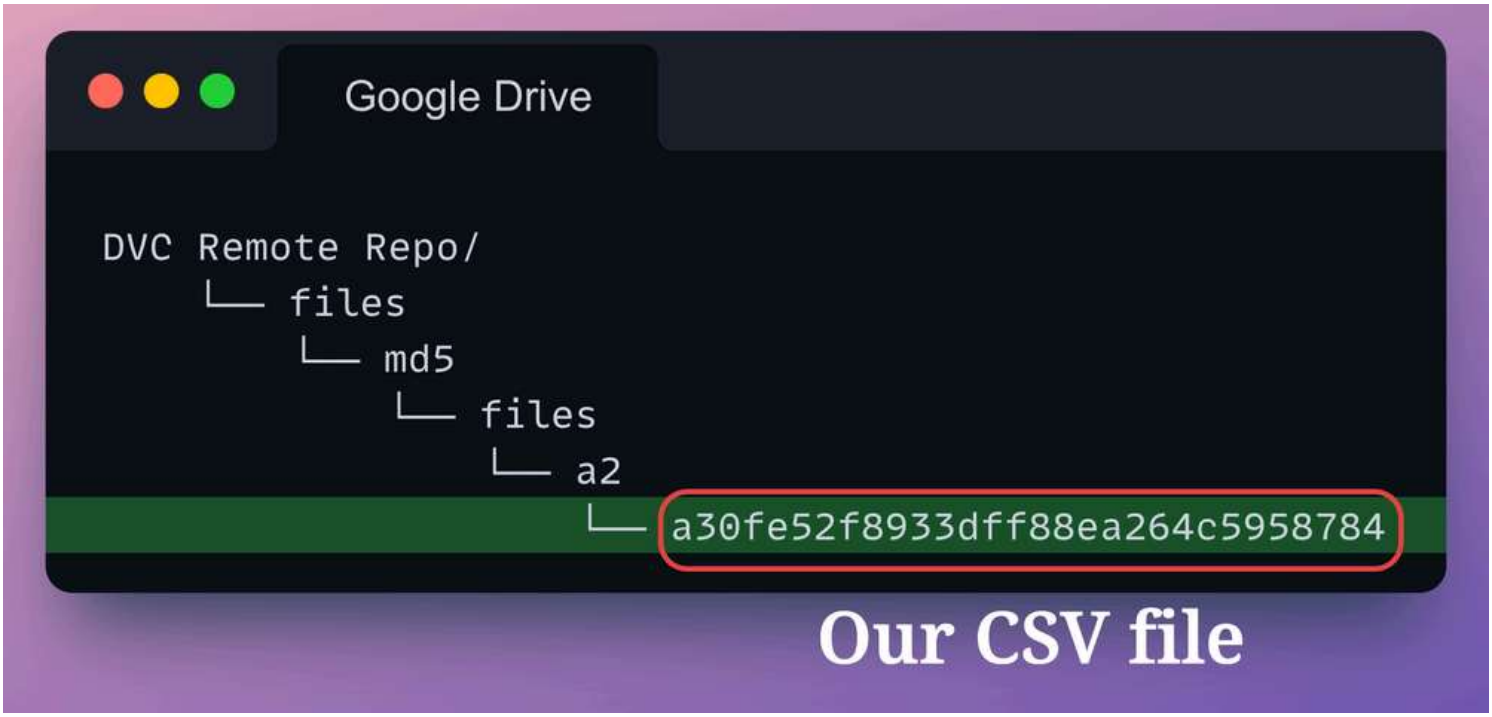
Along with `dvc push`, we also specify the alias (`myremote`) for our remote storage.

💡 After running the above command, you will be prompted with an authentication link in the terminal to establish the connection. Grad the link and paste it into the browser to authenticate. Also, you might be asked to install `dvc-gdrive`. Install it as follows: `pip install dvc-gdrive`.

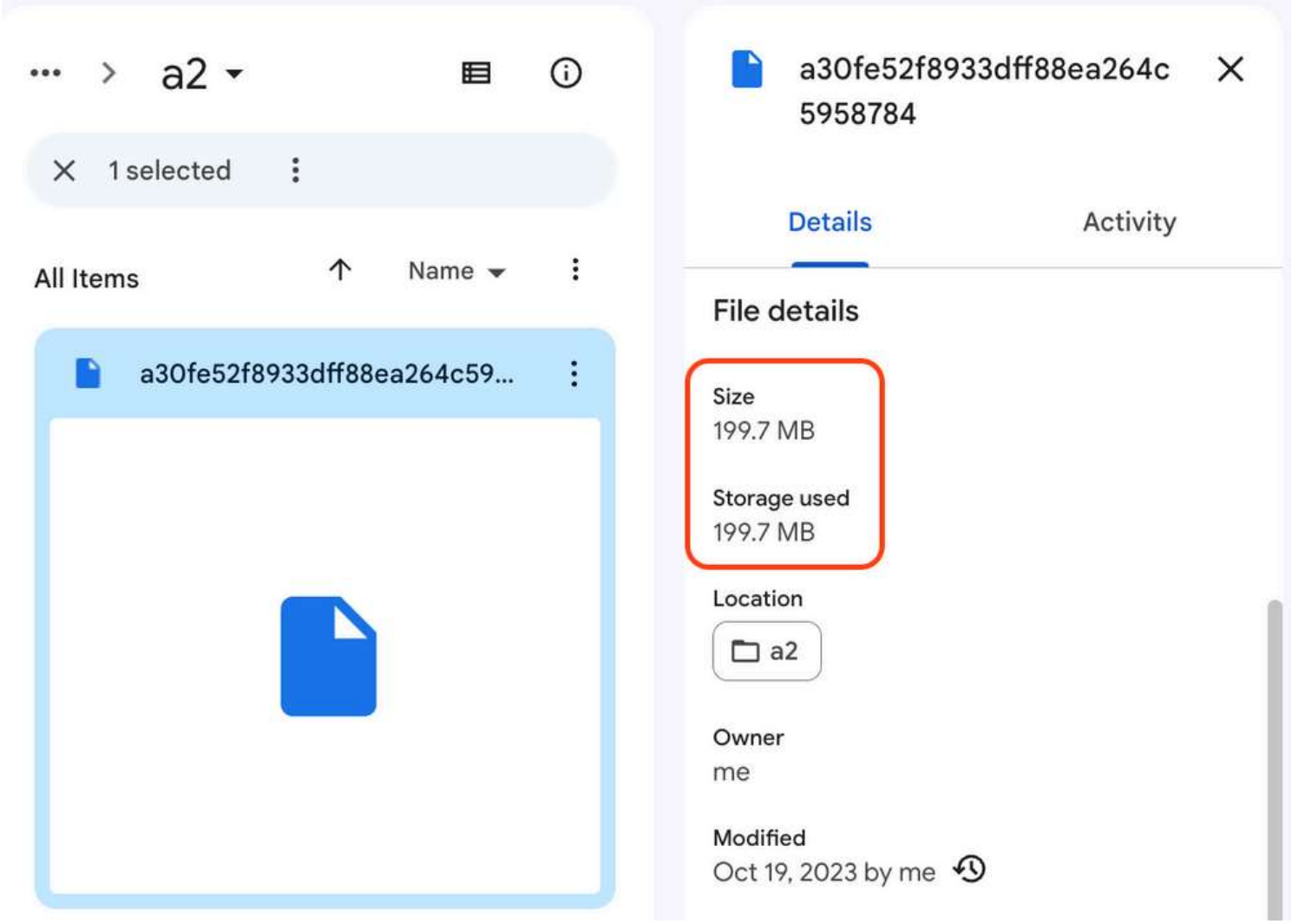
Once done, we notice a new folder in our remote storage.




Let's look at the directory structure of this our Drive folder.

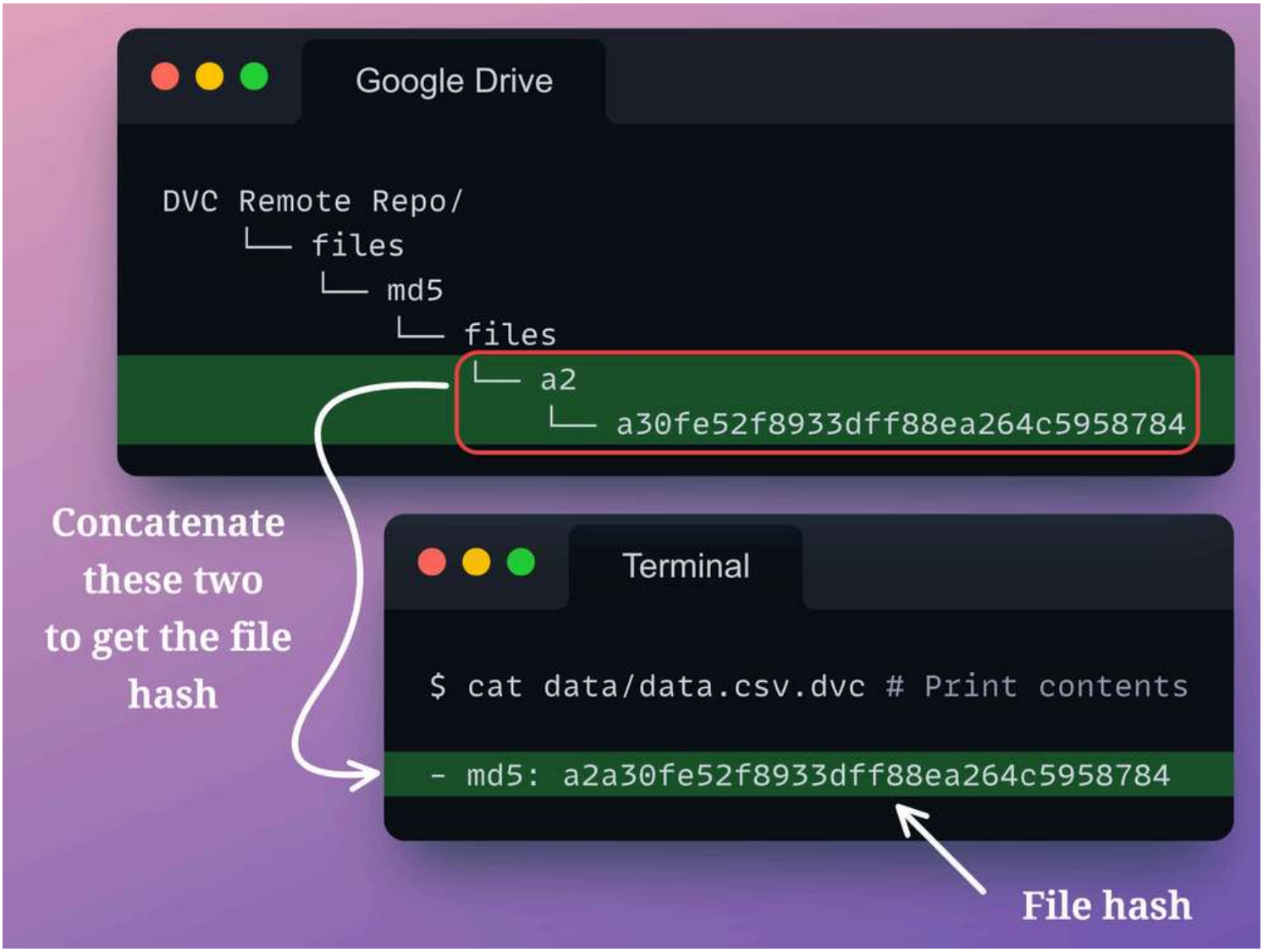



The file `a30fe52f8933dff88ea264c5958784` is our CSV file, and it takes up the exact same space:




Now, you might be wondering, what is that mysterious name?

But if you look closely, the `md5` hash of the dataset is the same as the concatenation of the folder enclosing the file `a2` and the name of the uploaded file.




So overall, DVC creates a neat structure to maintain data version control in our project.

- First, it creates a hash value of the dataset in the `.dvc` file.
- During data push, it ensures that the file name of the dataset sent to remote storage and the name of the folder it is enclosed in matches the file hash.
- Being lightweight, Git easily versions the `.dvc` file, which stores the hash value of different versions of the dataset.
    - Simultaneously, DVC ensures that the name of the file uploaded to remote storage stays coherent with the file hash.
- While bringing an uploaded dataset back to local storage from cloud, DVC can precisely identify which dataset is to be downloaded by looking at the hash value in the `.dvc` file.

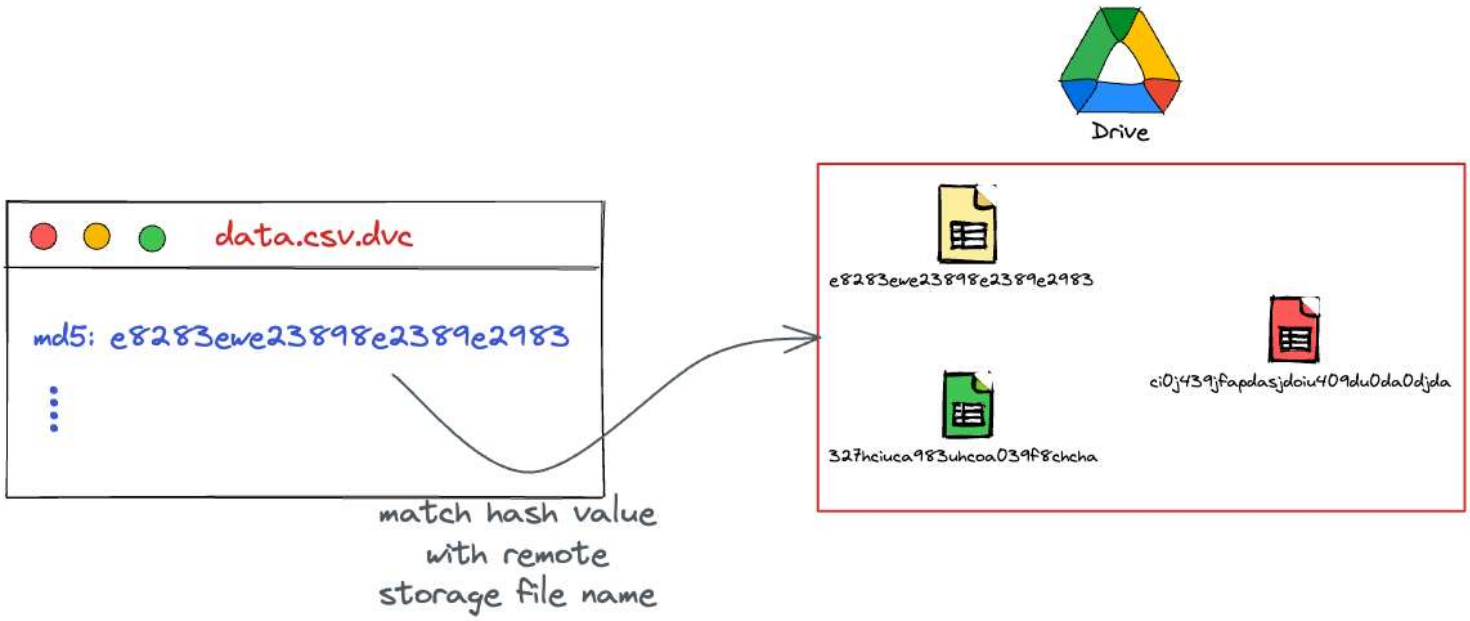
Pretty simple and straightforward, isn't it?

# Data pull with DVC

To reiterate, in the above discussion, we looked at how DVC smartly addresses the problem of versioning large files.

Instead of versioning the large files directly with Git, it instructs Git to version the hash file instead, which is lightweight.

Later, when doing a data pull from remote storage, DVC can look up the hash value stored in the `.dvc` file to the files pushed to remote storage.



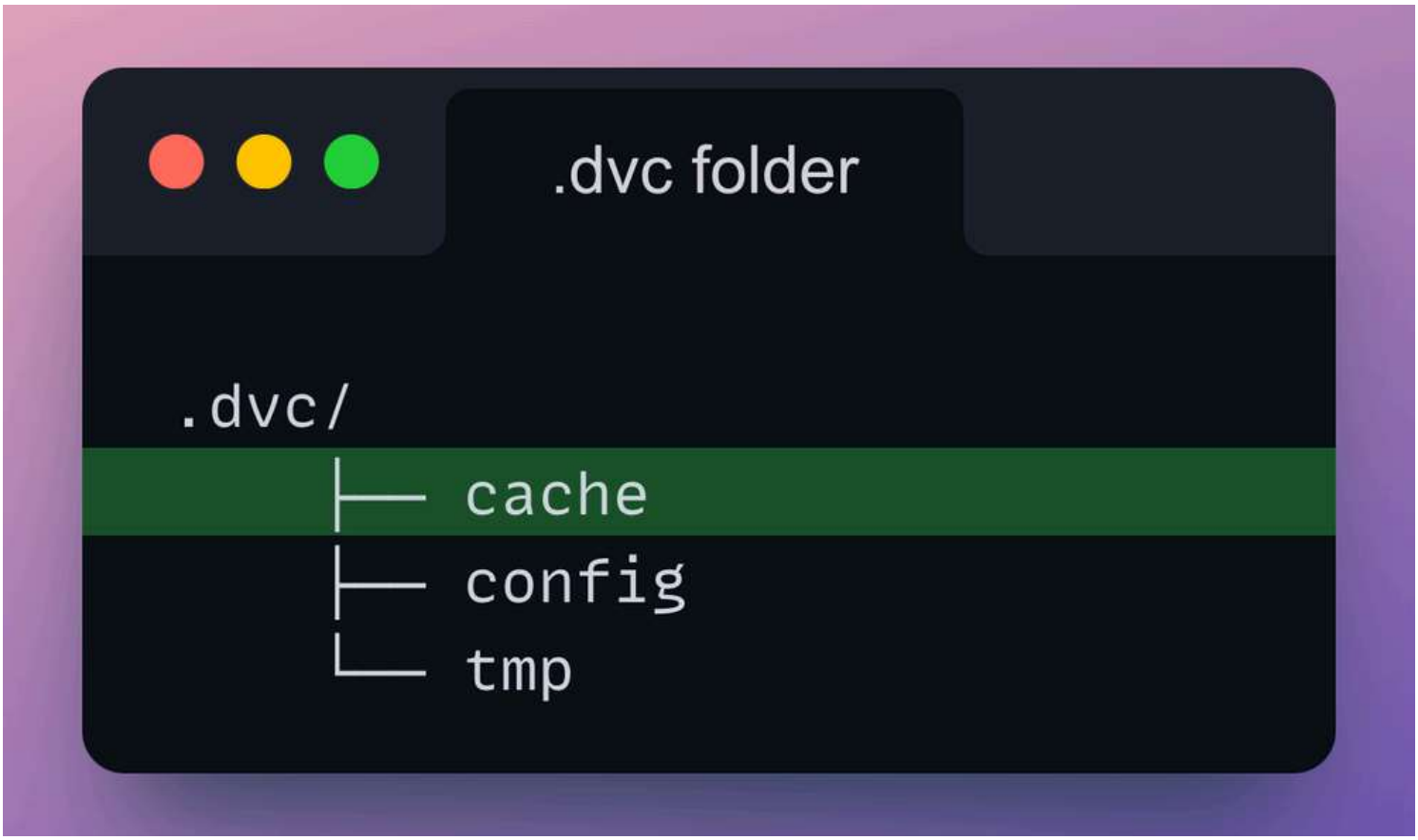

Let's see how that works.

Let's assume that the `data.csv` does not exist. In fact, let's delete it from our local project folder:

Just like we pull a codebase from the remote repository using `git pull`, we can pull a dataset from remote storage using `dvc pull`.

However, before doing that, we must also clear the DVC cache, which, to recall, is present in one of the five files (or folders) generated earlier with `dvc init`.

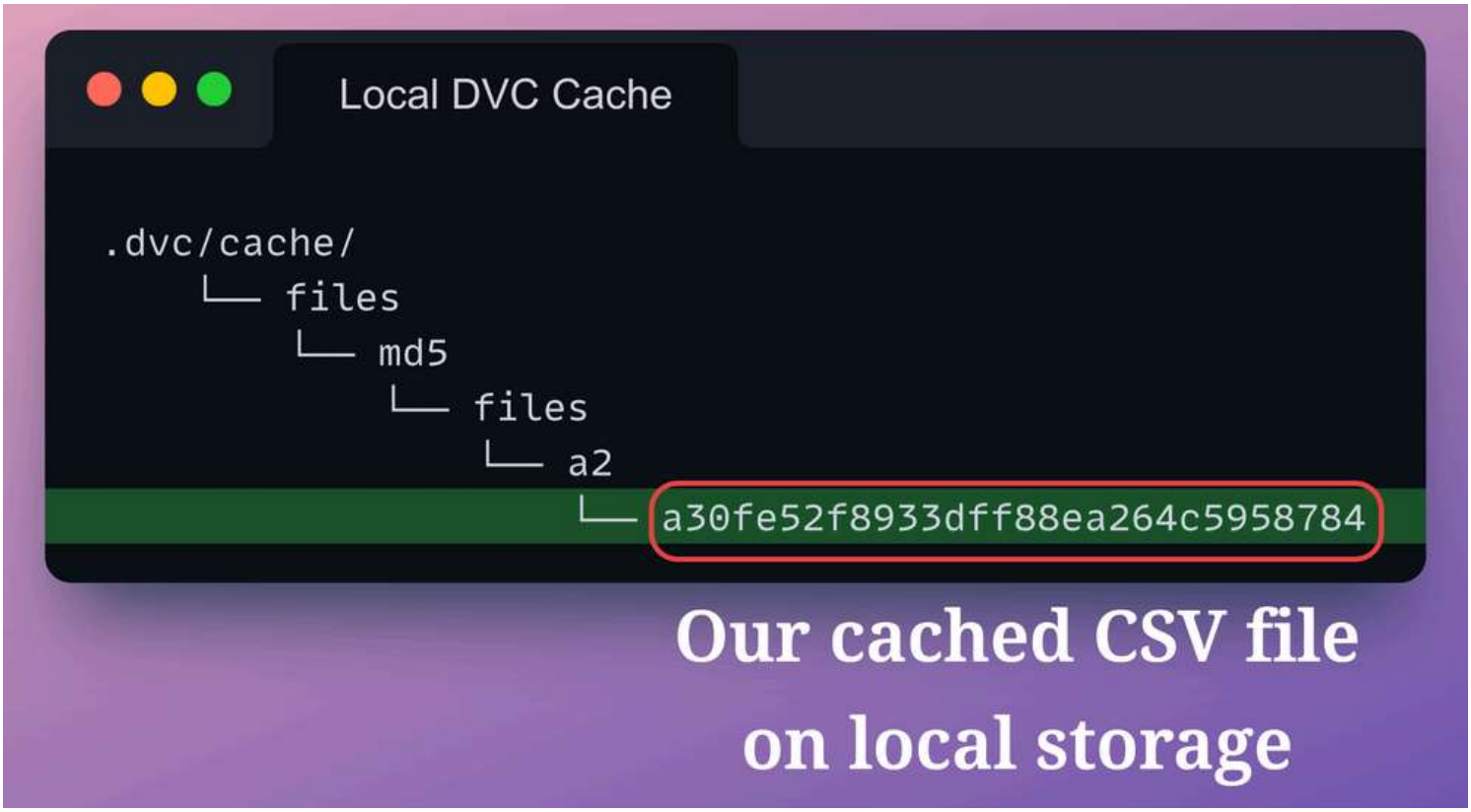
More specifically, the `dvc init` command creates a `.dvc` folder:




Within this folder, there is a `cache` folder, which, as the name suggests, caches the large files we add with `dvc add` and later push to remote storage with `dvc push`.

DVC creates this to ensure that we don't duplicate any dataset that we may have downloaded recently.

In fact, the directory structure of the cache folder is exactly the same as what we saw in the remote storage on Google Drive:




Let's delete this `.dvc/cache` folder as well:

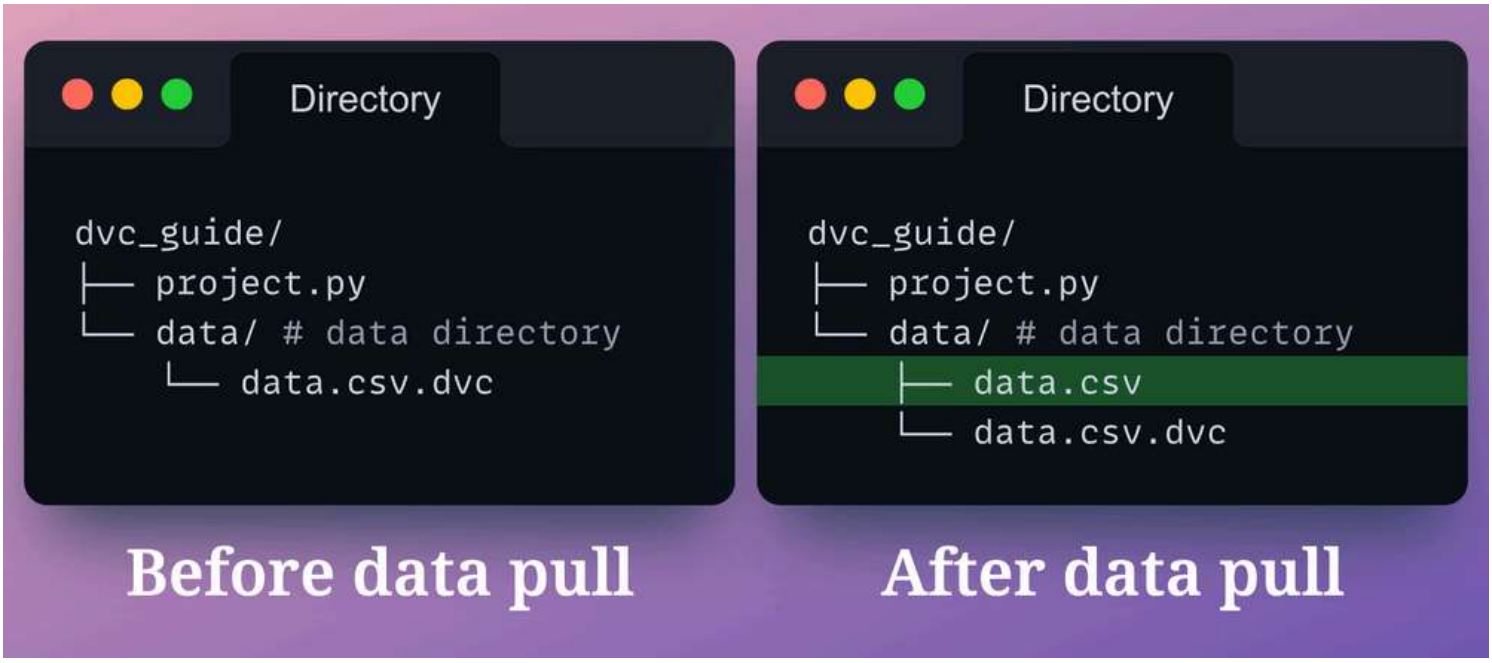
Now we are good to pull the dataset from remote storage using `dvc pull`:

Similar to what we did with `dvc push`, during `dvc pull`, we specify the alias for the remote storage to pull the dataset from.

- The `-r` option in this command is used to specify the remote storage to pull the dataset from.

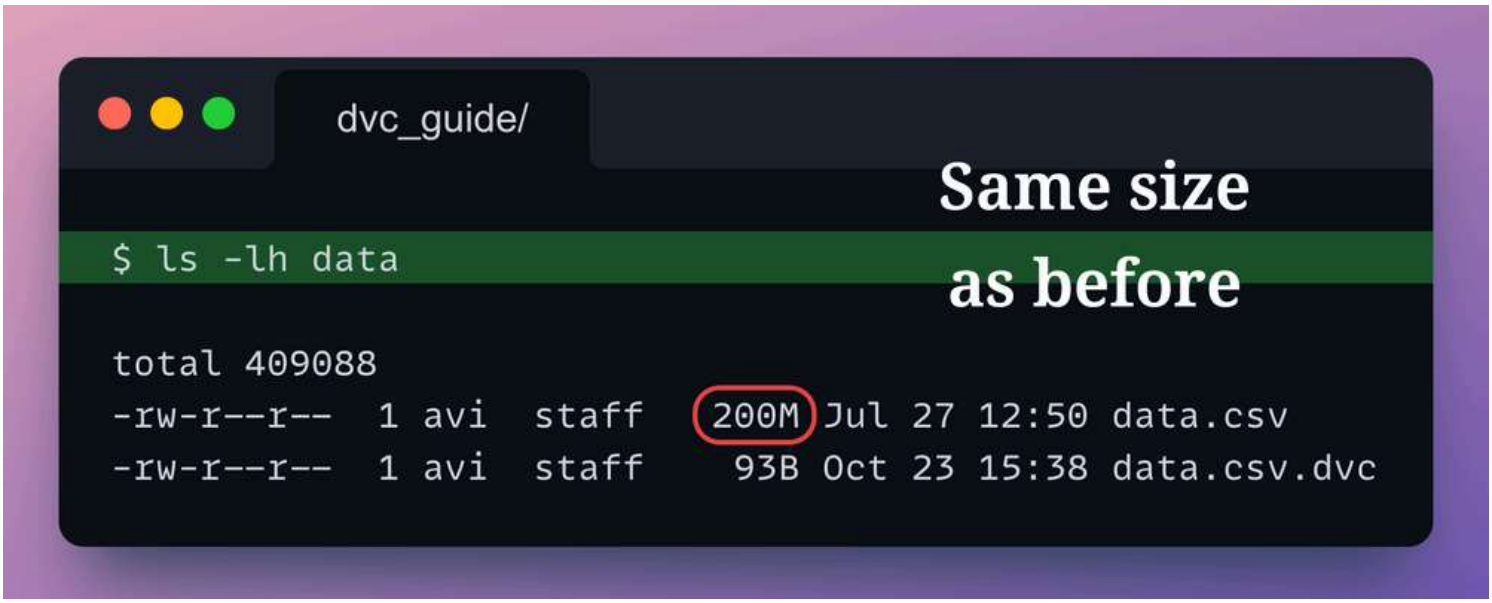
After pulling the dataset, DVC says that it has added 1 file.

Let's check if the data file has been added or not:



It sure did!

Let's check the file size to be sure that we have pulled the right dataset:




The file size is the same, which verifies that we have pulled the correct dataset.

So now we know how to push and pull to and from the remote storage location.

# Version control datasets

Next, let's look at how we can version control our dataset.

In other words, we shall change our dataset and use DVC to version the dataset and Git to version the hash file.

We shall also upload the dataset to remote storage.

Let's make a slight change to the data file `data.csv`.

First, let's check the number of lines in this file:

Let's say we only want to keep the first `10000` lines of this dataset. We can do this as follows:

Let's run `dvc status` to check if DVC noticed this change or not:


As shown above, DVC did identify the change.

Before we proceed ahead, here's a question for you:

**What will be the output if we run `git status` at this stage, i.e., after only modifying the `data.csv` file?**

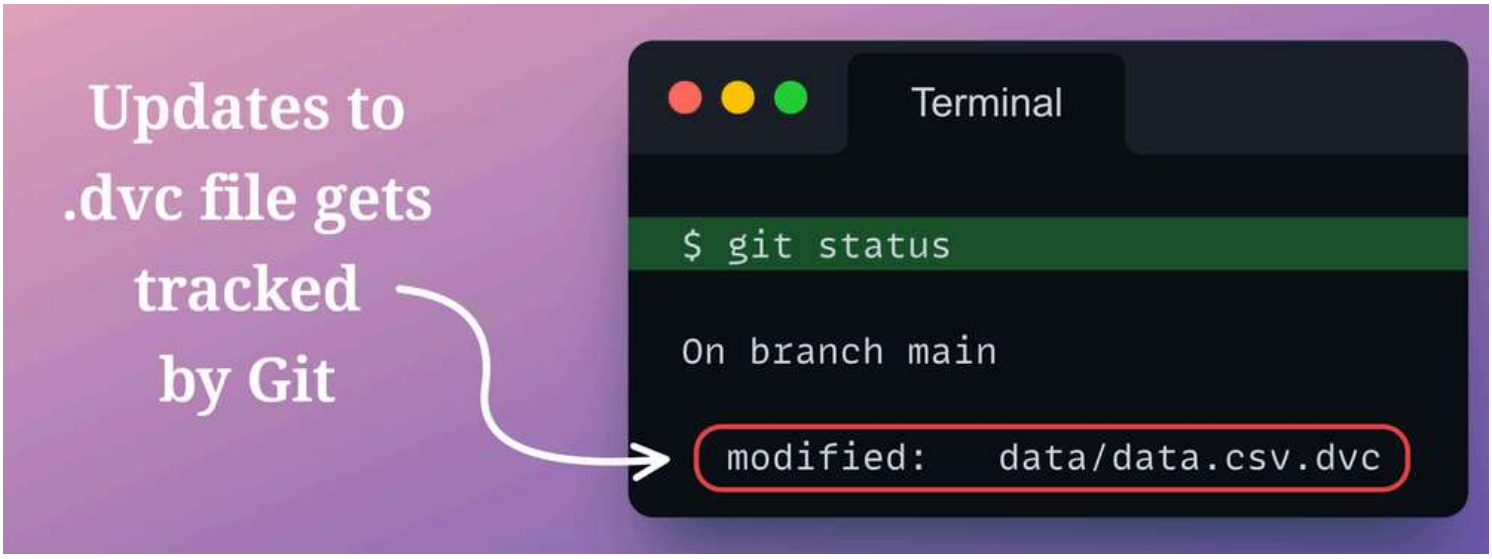
Now that we have made some changes to the `data.csv` file, and DVC has also prompted us with those changes with the `dvc status` command, let's add the file to the DVC staging area with `dvc add` command:

As expected, the `data.csv.dvc` file (the file that holds the hash value) gets modified as well. That is why DVC asks us to stage the changes made to this file with Git.

Here's another question for you:

**What will be the output if we run `git status` now, i.e., after staging the `data.csv` file with DVC?**

Let's verify this by running `git status` command:



As depicted above, Git has identified changes to the `data.csv.dvc` file.

We can also verify this by comparing the current contents of `data.csv.dvc` file with what we had earlier:


As depicted above, the hash value has changed.

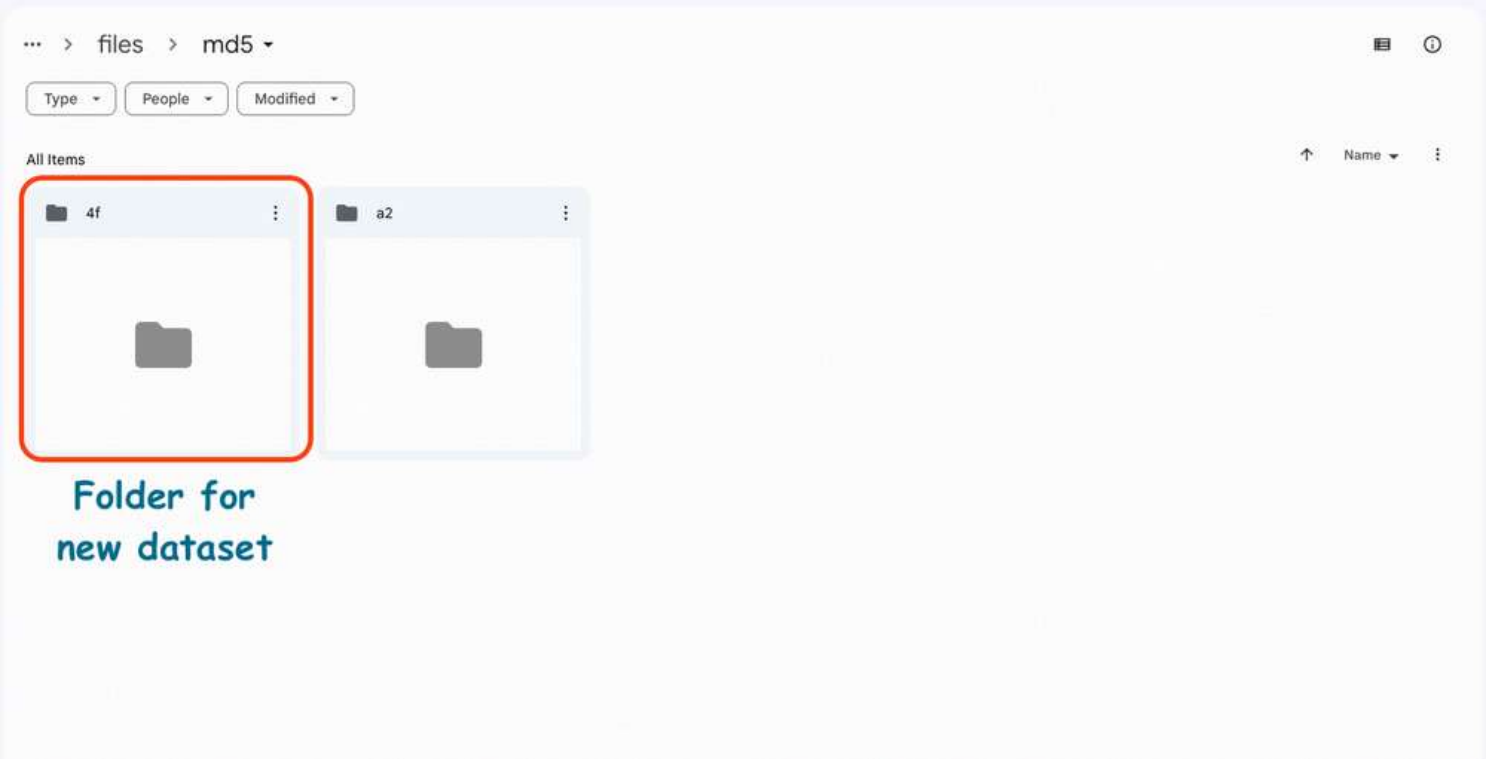
Let's add the `data.csv.dvc` file to the staging area and commit the changes to the local git repo:

Now, we can run `dvc push` to upload this dataset to remote storage:

Done!

We have successfully pushed our reduced dataset (the latest version) to remote storage.

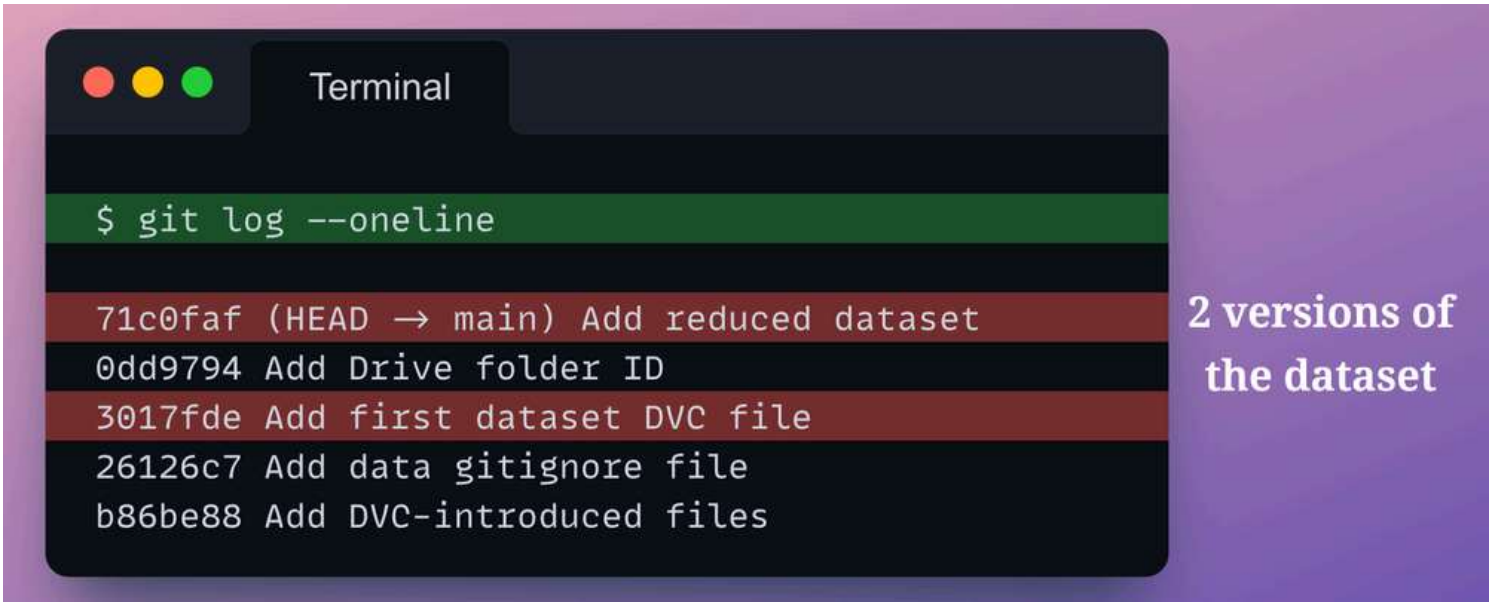
If we check our Google Drive folder, we see a new folder `4f`, the name of which is the same as the first two characters of the hash value of the reduced dataset.



The directory structure of Google Drive is now:


Now, to summarize, we have had two different versions of the dataset in two different commits.

We can verify this from our git log:



Now, let's say we want to switch to a previous version of the dataset.

In the above visual, we can see that we updated the dataset in the latest commit, and there are two different versions of this dataset.

- Commit ID: `71c0faf`.
- Commit ID: `3017fde`.

To go back to the "Add first dataset DVC file" state, we can use `git checkout`.

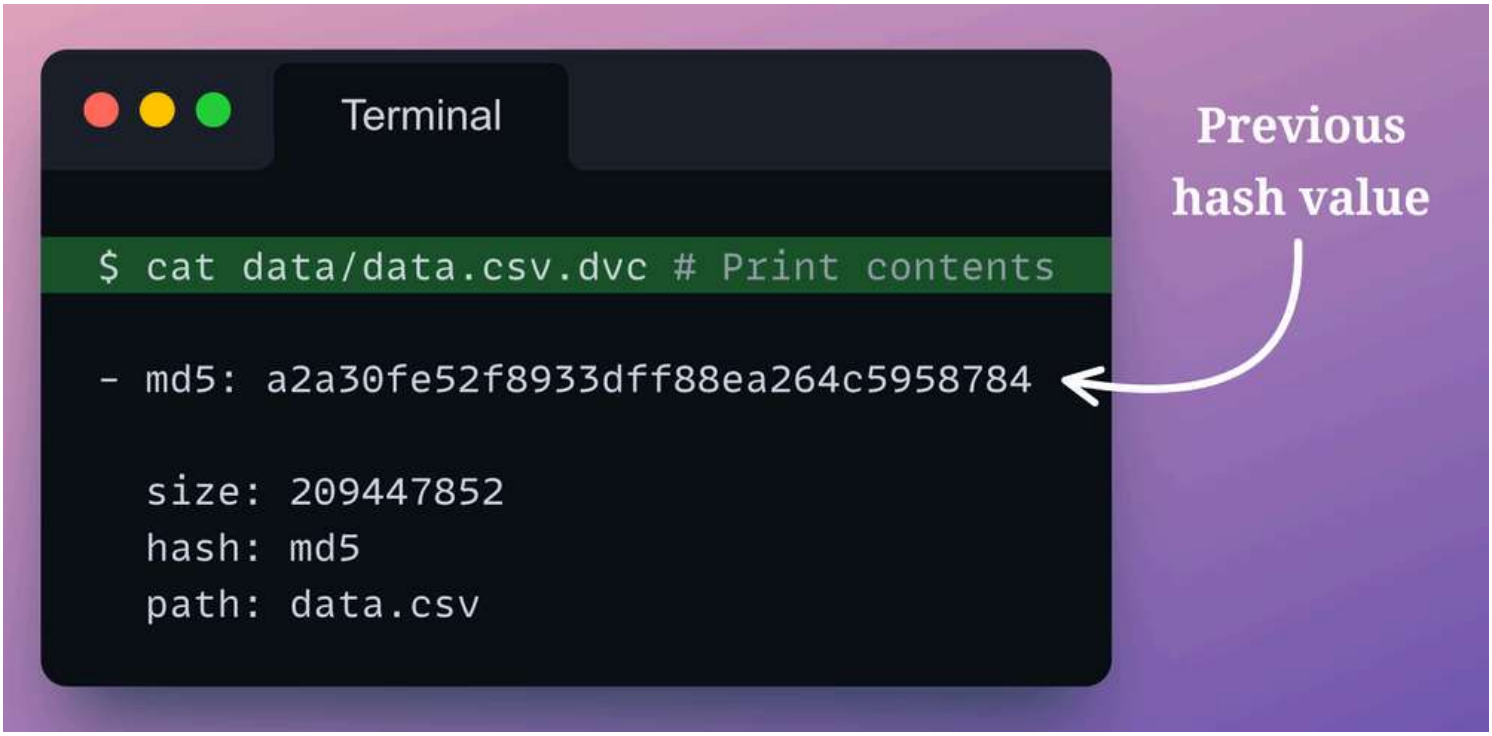
> 💡 `git checkout` is a command in Git that allows us to switch between different versions of a target entity such as files, commits, and branches.

To checkout files from a previous commit, we can reference the commit using its 7-letters long alphanumeric commit ID.

In our case, we can reference the relevant commit with the commit ID: `3017fde`.

Here, the reason we are not checking out the `data.csv` file is that the data itself is not being tracked by Git. Instead, it's the `data.csv.dvc` file that is being tracked by Git.

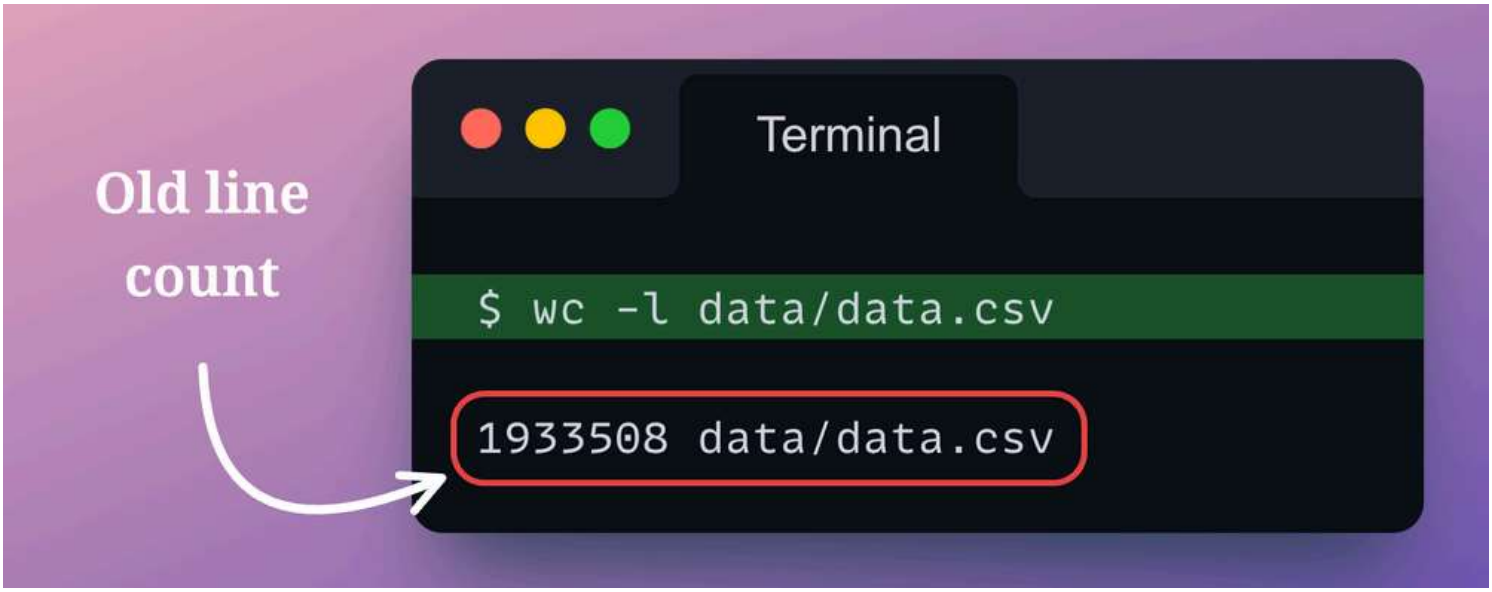
After running the above command, we can see that the contents of `data.csv.dvc` have changed:



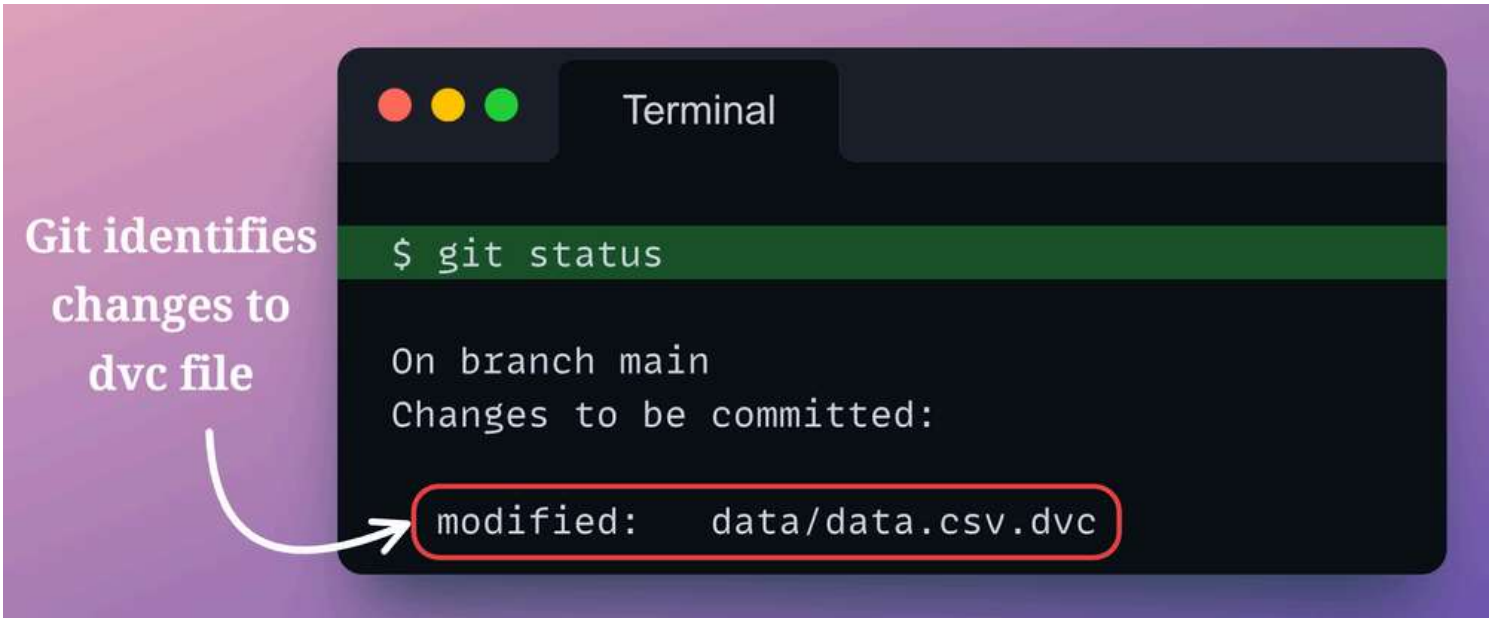

The hash value is the same as what we had earlier – one that started with `a2`.

After running the above command, we run `dvc checkout`, which is used to get our data back from the cache:

Yet again, we can indeed verify that the `data.csv` file has been updated by printing the number of lines in this file:



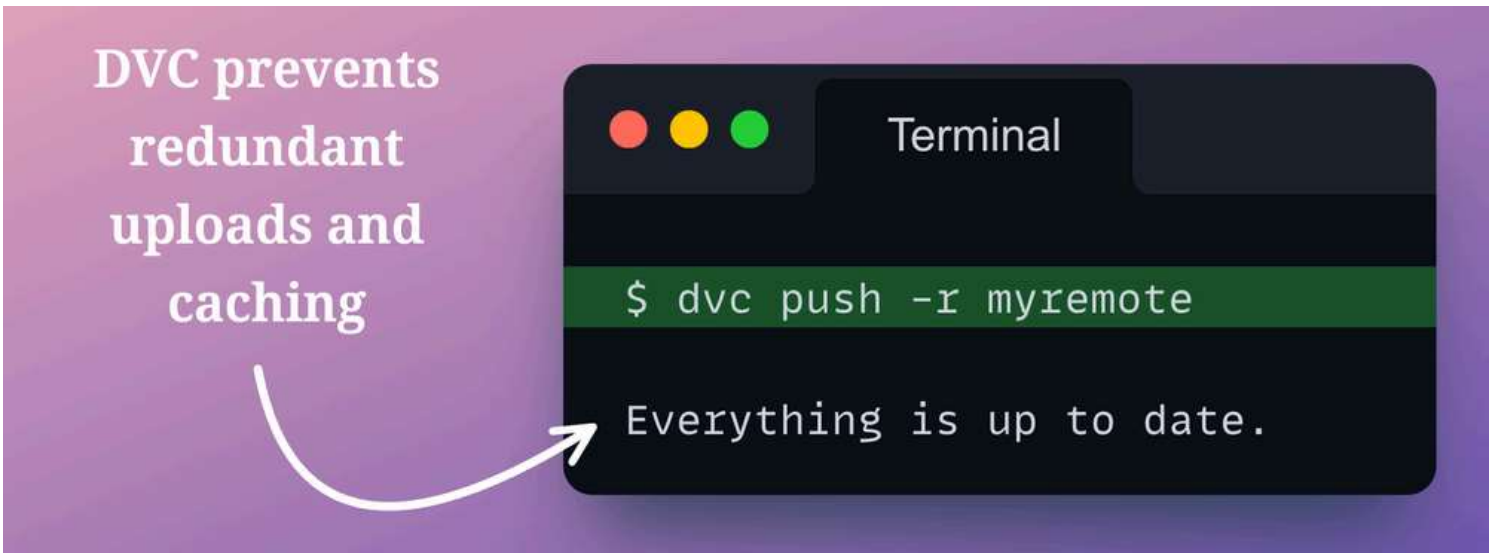

Next, as the `data.csv.dvc` file was updated in this checkout, Git would have noticed that too. We can verify this with `git status`:




Let's commit these updates real quick with `git commit`:


Here, as we have reverted to an older version of the dataset, which is already in the cache and in the remote storage, we don't have to run `dvc add` and `dvc push` again.

In fact, DVC is so smart that even if we run `dvc push`, it would identify that the exact version of this dataset already exists, and it does not have to version that again:

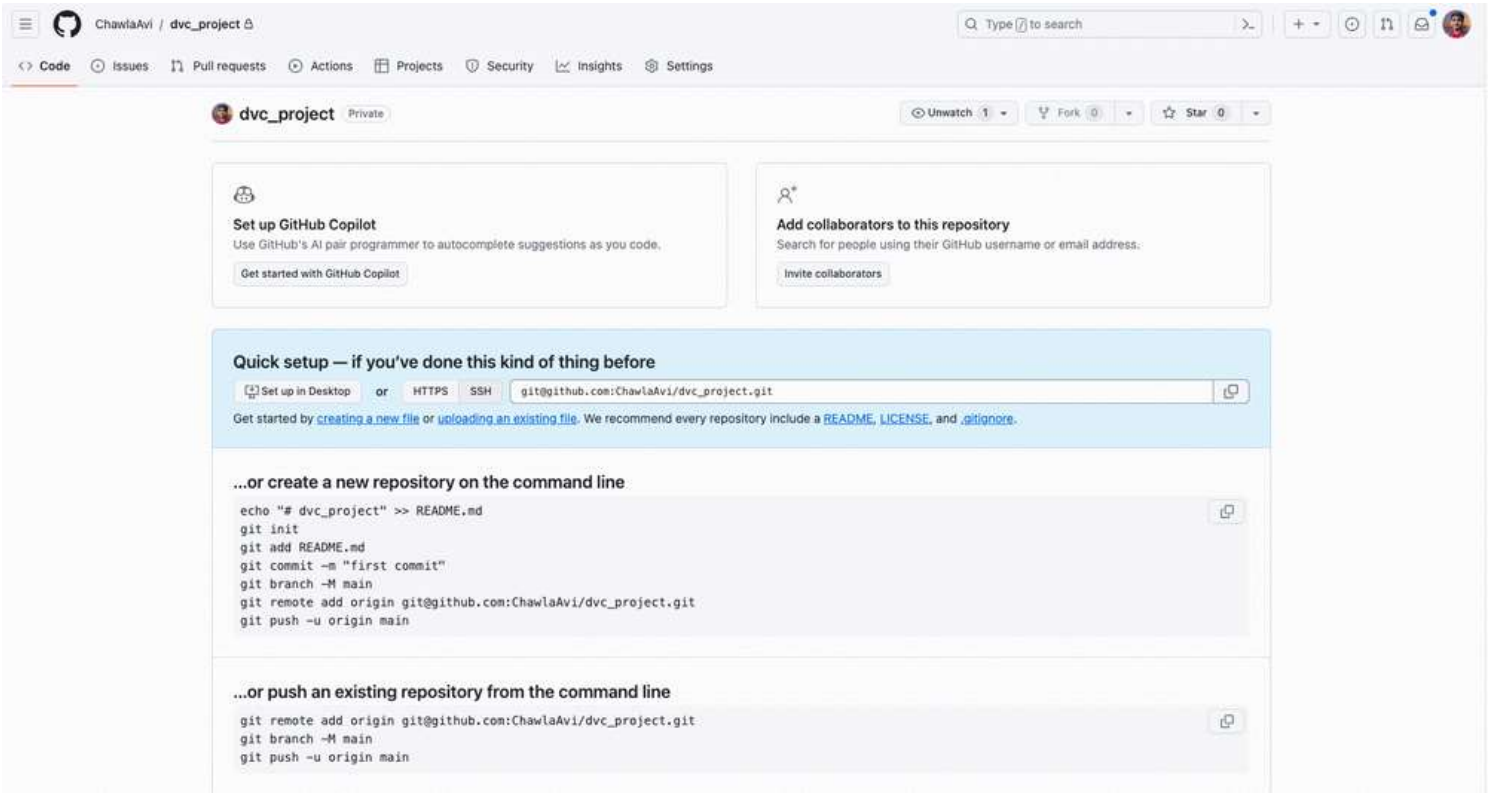



Pretty cool, isn't it?

# Push to GitHub

Next, let's push our local Git repository to a remote GitHub repository and see how things look over there.

For this purpose, we have created a new GitHub repository `dvc_project`:

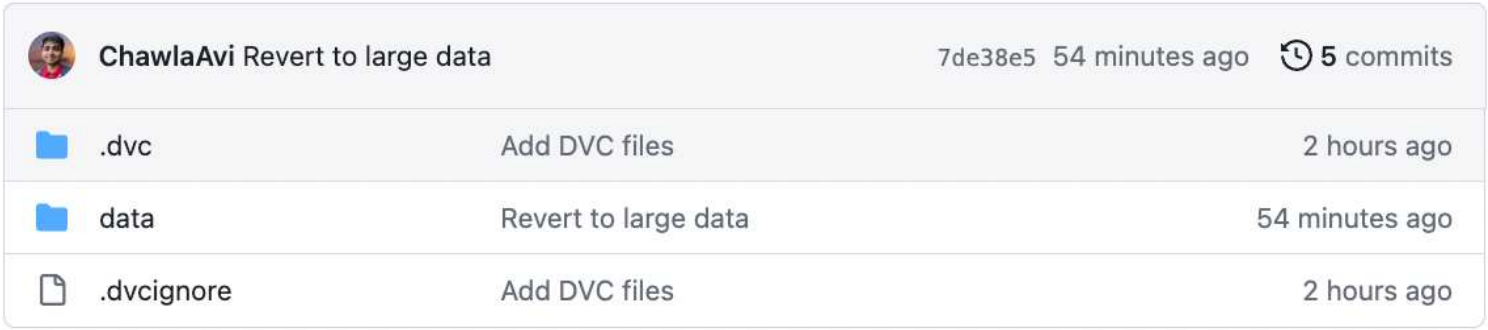



As we had already initialized our git repository earlier and made some local commits, we are ready to host our local git repository here.

To do this, we run the following Git commands:

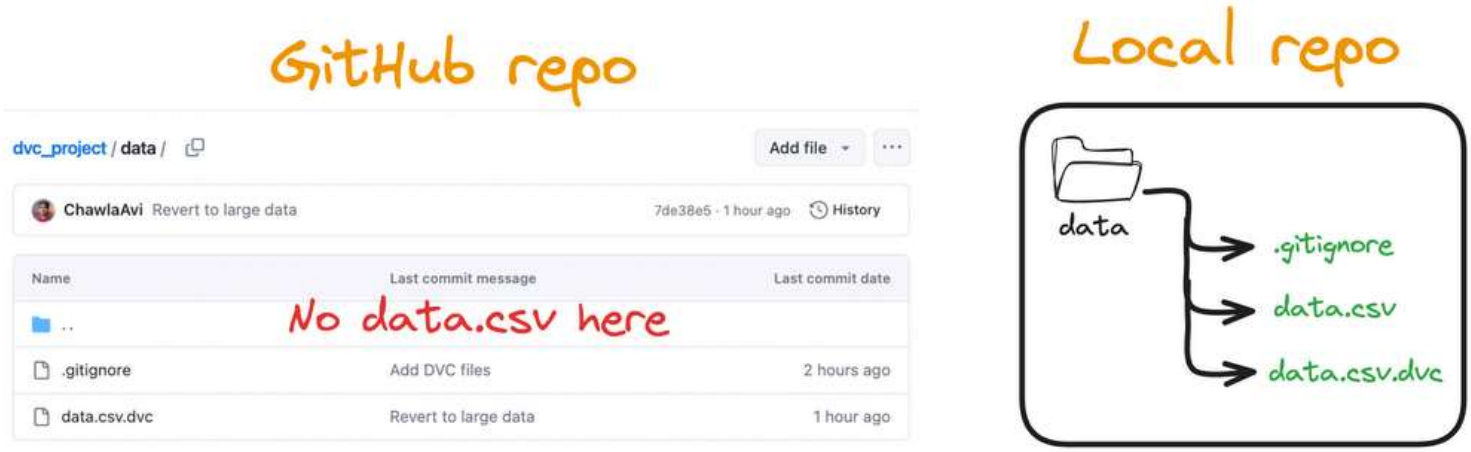
Done!

Heading to our GitHub repository, we notice that it has the following files and folders:




The latest commit is “Revert to large data,” which is precisely what we did recently.

After opening the `data` folder, we notice that there is no `data.csv` file here, but it is present in our local repository, which is desired.




To reiterate, the way this whole system works is that instead of tracking the dataset directly, we ask Git to keep track of the `data.csv.dvc` file, which holds a unique hash value for our dataset.

DVC can use this hash value to coordinate between a git repository and the remote data storage during push and pull operations.

Cool stuff, right?

# DVC hacks and misconceptions

Now that we know:

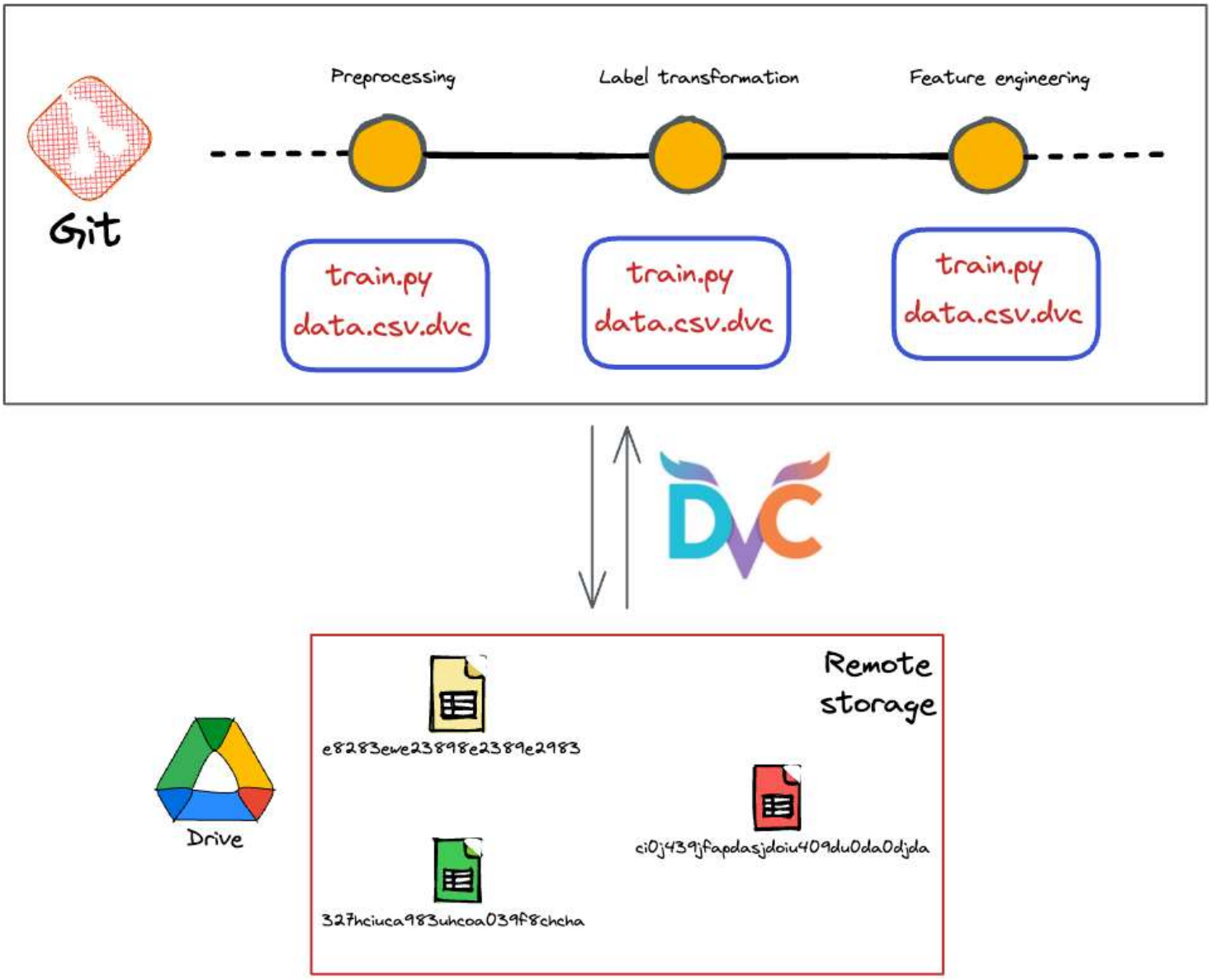
- how to track and upload files to the DVC remote
- how to configure DVC with git
- how DVC works end-to-end, etc.,

...let's take a closer look at DVC's internal working, hacks, and some misconceptions.

# DVC is not a version control system

At this point, you might have started to realize that DVC is "technically" not a version control system, which is correct.

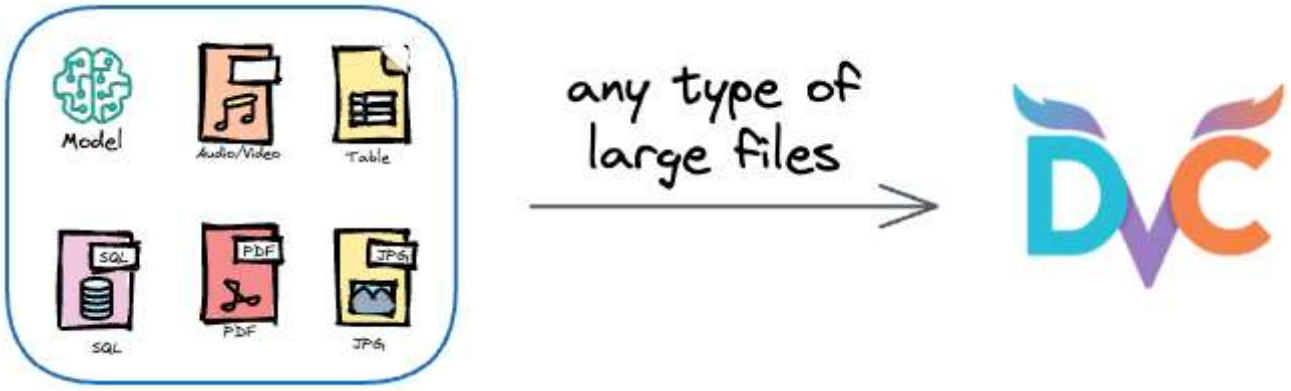
Instead, like before, it is git that is doing the version controlling of the hash file of our dataset. The role of DVC is to help us seamlessly coordinate between remote storage and Git repository.




DVC helps us extend git version control to files that we want to keep outside of git — things like large datasets, model files, etc.

So technically, the terms “version control” in DVC do not strictly indicate version controlling.

In fact, even the term “Data” in DVC does not mean that the tool is limited to versioning datasets.

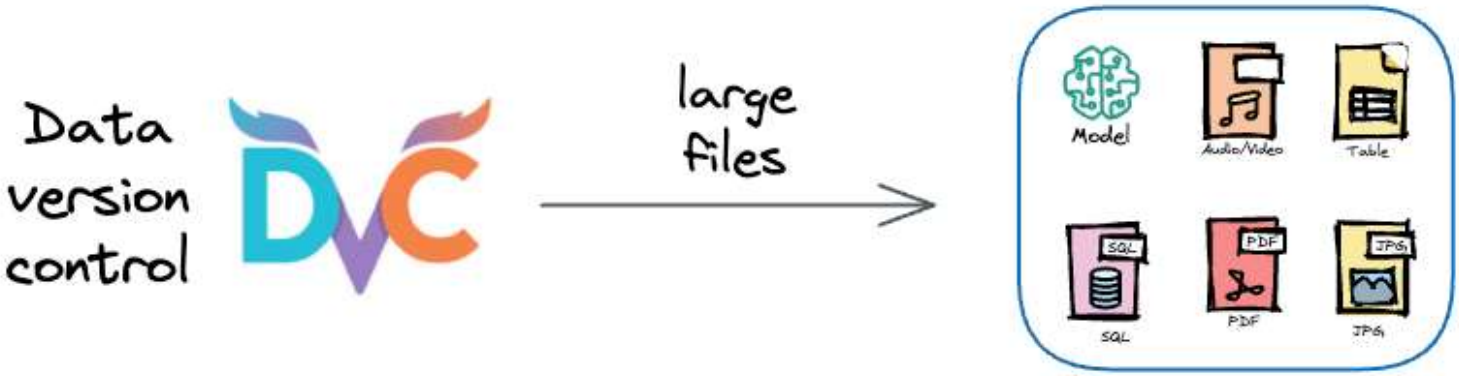



Instead, we can use it to version any file type, which may include datasets, model binaries, images, audios, videos, etc.

What's more, it is not that DVC is just limited to machine learning applications. As long as you want to version some large files in remote storage, DVC can be immensely useful.

## Configure DVC cache

In the entire demonstration above, DVC remote can be considered analogous to GitHub.

Just like we store scripts in a remote GitHub repository, we store large data and model files in DVC remote storage.

When we want to commit any changes to a Git repository, we first add those files to the staging area using `git add`.

Similarly, when we want to commit large files to remote storage, we first add them to the staging area of DVC, which is called the **DVC cache**. This is done when we run the `dvc add` command.

This is how the process works end-to-end:

- When we initialize a DVC repository using `dvc init`, DVC creates a `.dvc/cache` directory.
- After that, whenever we run `dvc add`, DVC copies the files to its staging area — **the DVC cache**.

**As you may have guessed, this leads to file duplication.**

Essentially, every file staged using `dvc add` will be available in two locations:

- The original location (provided it was not deleted).
- The cache folder.

If you are wondering why DVC unnecessarily introduces redundancy, this design is intentional.

It does this because, as we saw above, by maintaining a local copy in the cache, DVC can help you avoid downloading datasets that are already available. This also saves bandwidth and time.

But the problem is that if we go by the natural design of DVC cache, it is not wise for every team member to maintain their isolated cache.

Typically, data teams have a shared storage for all team members.

So just like we can configure the location of the remote storage (Google Drive, Amazon S3, etc.), we can configure the location of the cache.

**Simply put, every team member can configure their DVC cache such that it points to a shared/common location.**

Earlier, we saw that the cache folder is located in `.dvc/cache`.

We can configure DVC's cache location as follows:

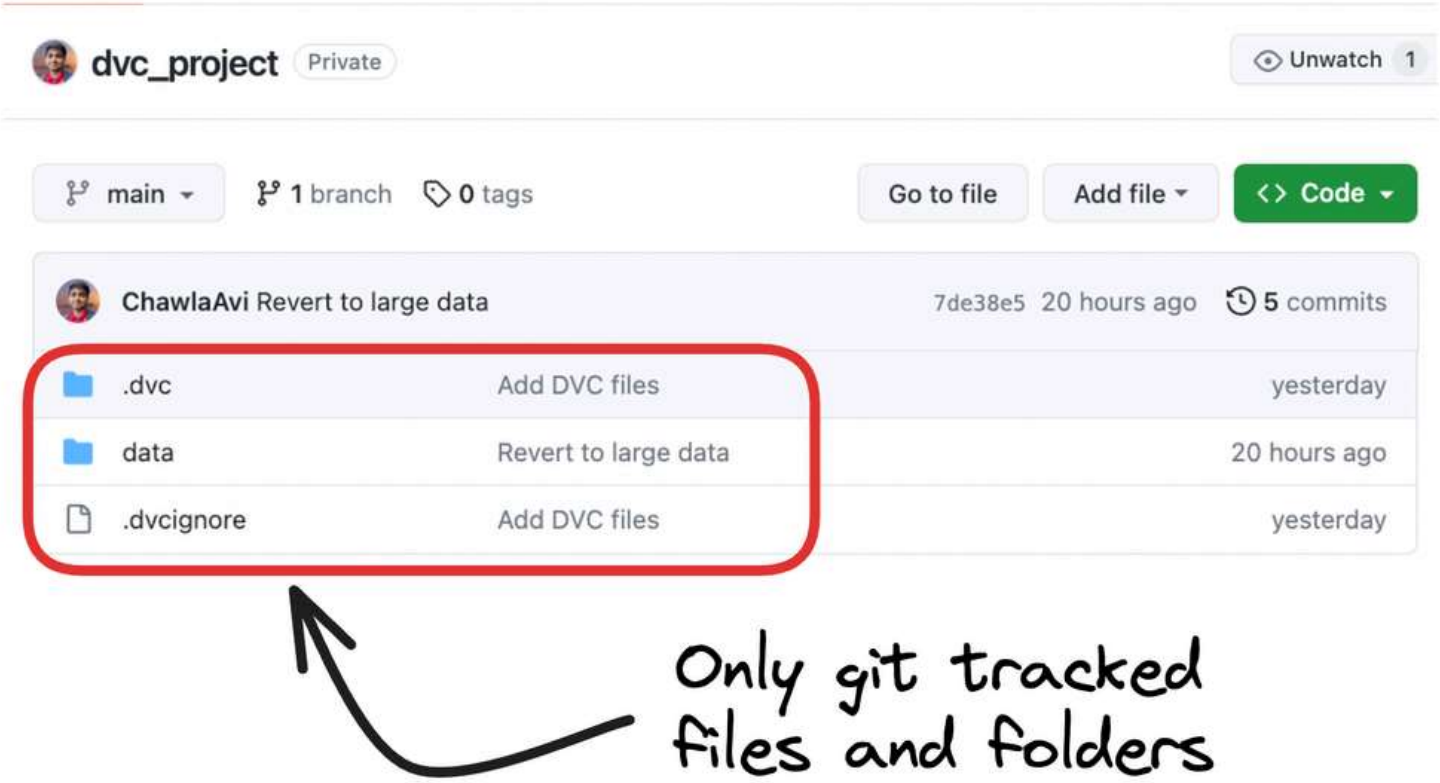
💡 During this process, the administrator of the shared location must ensure that all team members are granted read/write access to the shared location.

# Determine DVC-tracked files

As we saw above, in any project, there can be two types of files:

- Git tracked files
- DVC tracked files

But the files pushed to a GitHub remote repository are only those files that are being tracked by Git:




What if we want to know the DVC tracked files as well in a project using its remote repository?

We can do this by using the `dvc list` command:

As depicted above, in the `dvc list` command, we pass the GitHub repository and specify the name of the folder we want to check DVC tracked files in.

The command tells us everything in the folder and also includes the `data.csv` file, which is actually in cloud storage.

So that's how we can list all DVC-tracked files associated with our project.

# Download data file

So now that we have listed the files, what if we want to download our data file to local storage?

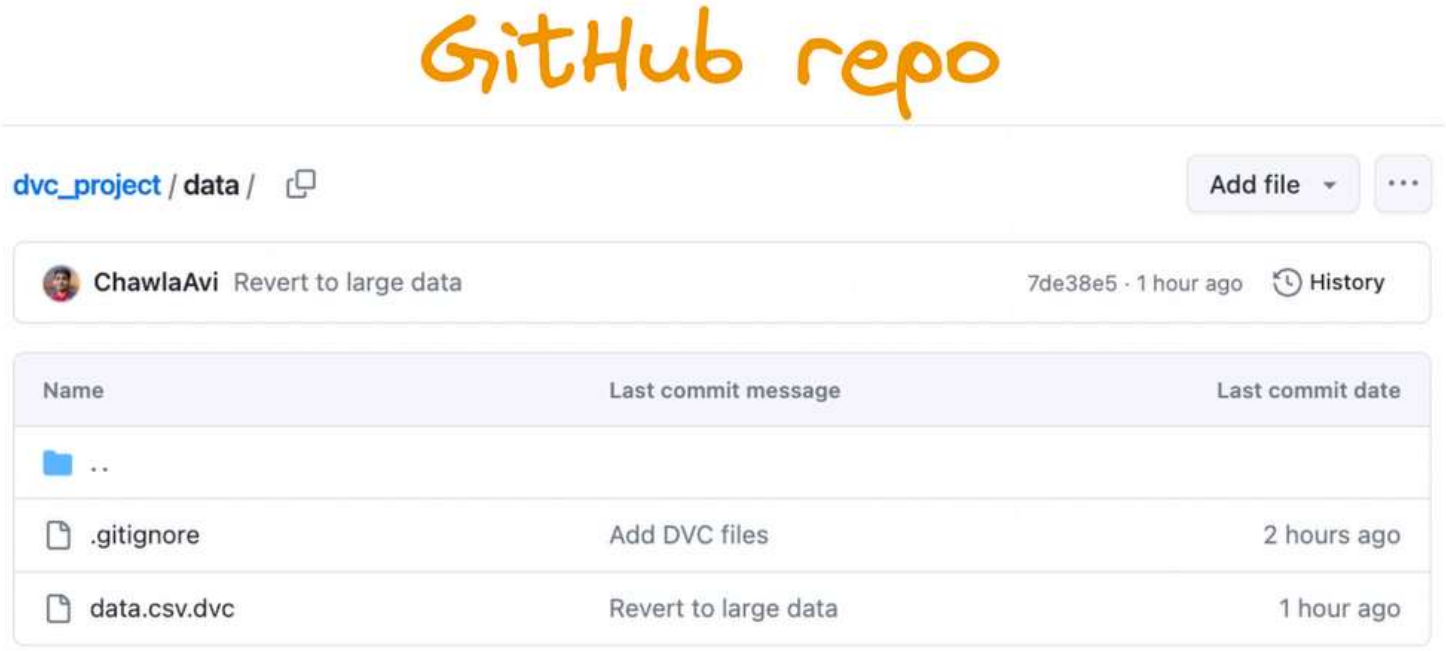
But the problem is that they are not available directly in the GitHub repository.

DVC simplifies this process too.

To download any dataset whose `.dvc` file is available in the remote repository, we can use the `dvc get` command.

Just like commands this `wget`, the `dvc get` command lets us download a file or directory tracked by DVC into the current working directory using its hash file.

For instance, say we want to download the `data.csv` file we had earlier. However, this is missing from our GitHub repo, as shown below:



Instead, we have the `data.csv.dvc` file here, which we can see in the `data/` folder of our GitHub repository.

To get the corresponding dataset, we can use `dvc get`.

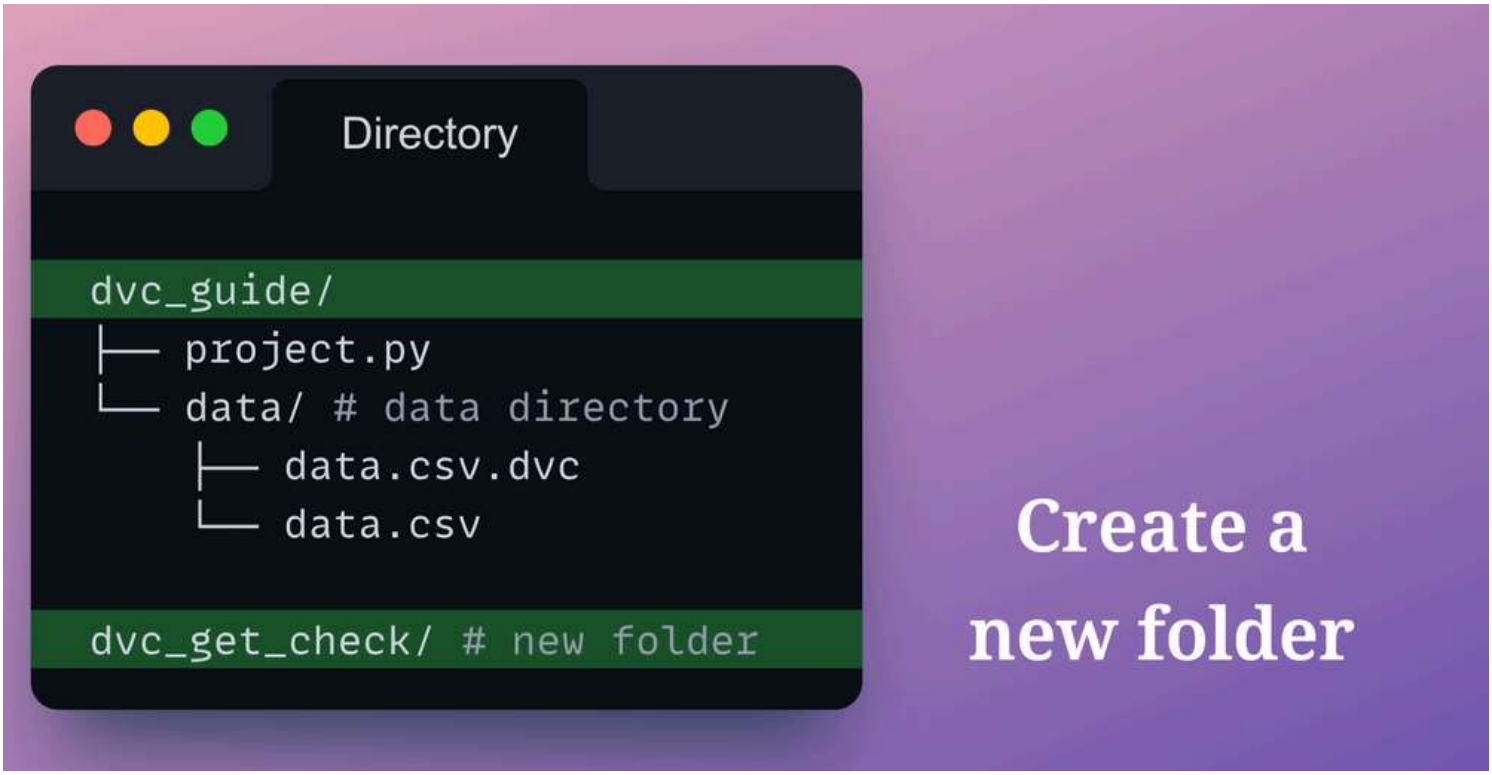
But before doing that, we must understand something.

💡 `dvc get` is different from `dvc pull`.

- `dvc get`:
  - The `dvc get` command is primarily used to retrieve files or directories tracked by DVC and Git and place them in our current working directory.
  - The current working may not necessarily be a git repository. In other words, it could be any repository where you want to download some dataset.
- `dvc pull`:
  - As we saw before, `dvc pull` is used for pulling data from a remote storage location (like a remote server or cloud storage) into our local **DVC-managed** repository.
  - It works in conjunction with our DVC project and associated data remotes.

To do this, move to any location outside the local DVC project directory we have been working in so far.

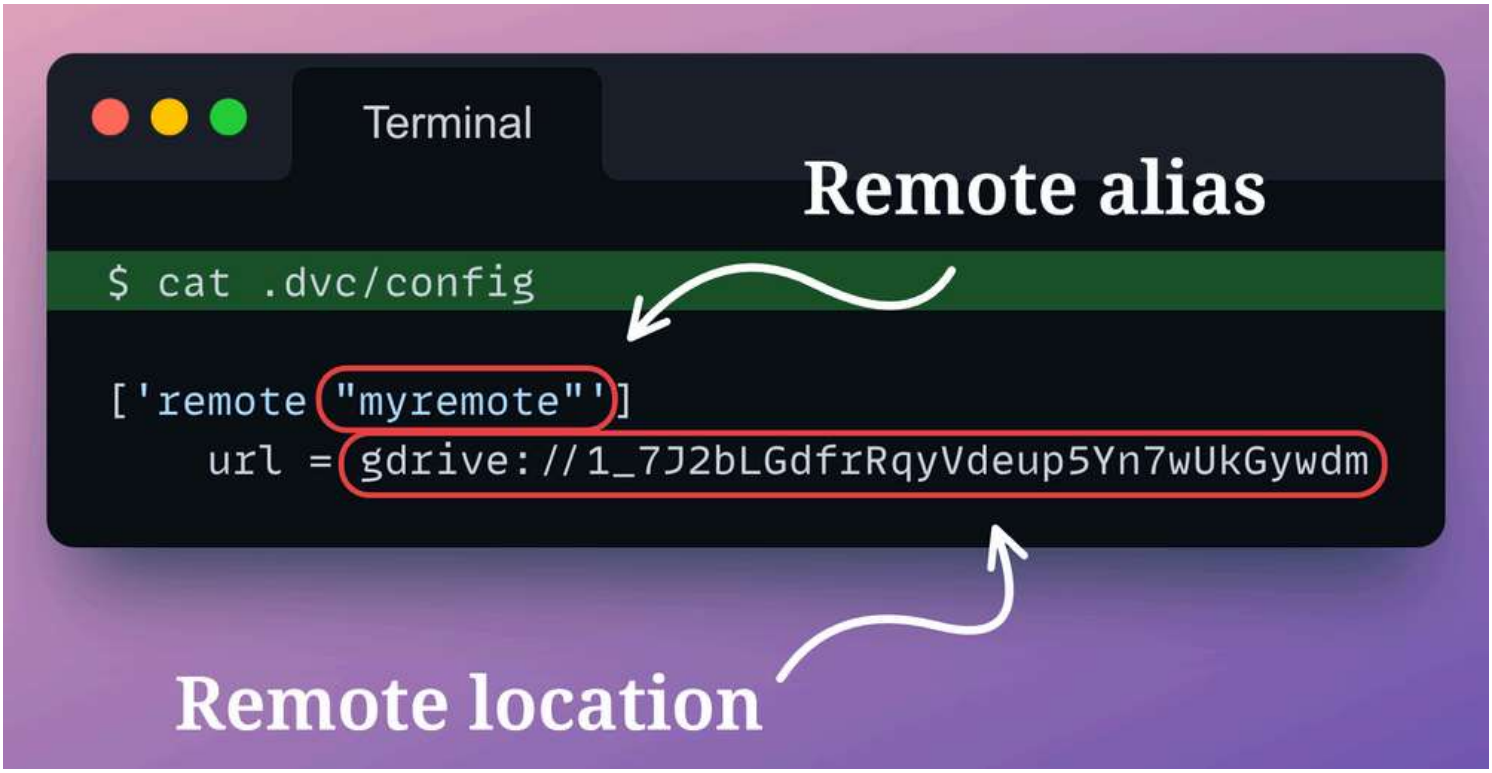
Let's assume we have created a new folder – `dvc_get_check`.




We must move to this directory in the terminal using `cd`.

Also, as discussed earlier, in a project, there can be multiple remote locations where data/models (or, in general, large files) may be stored.

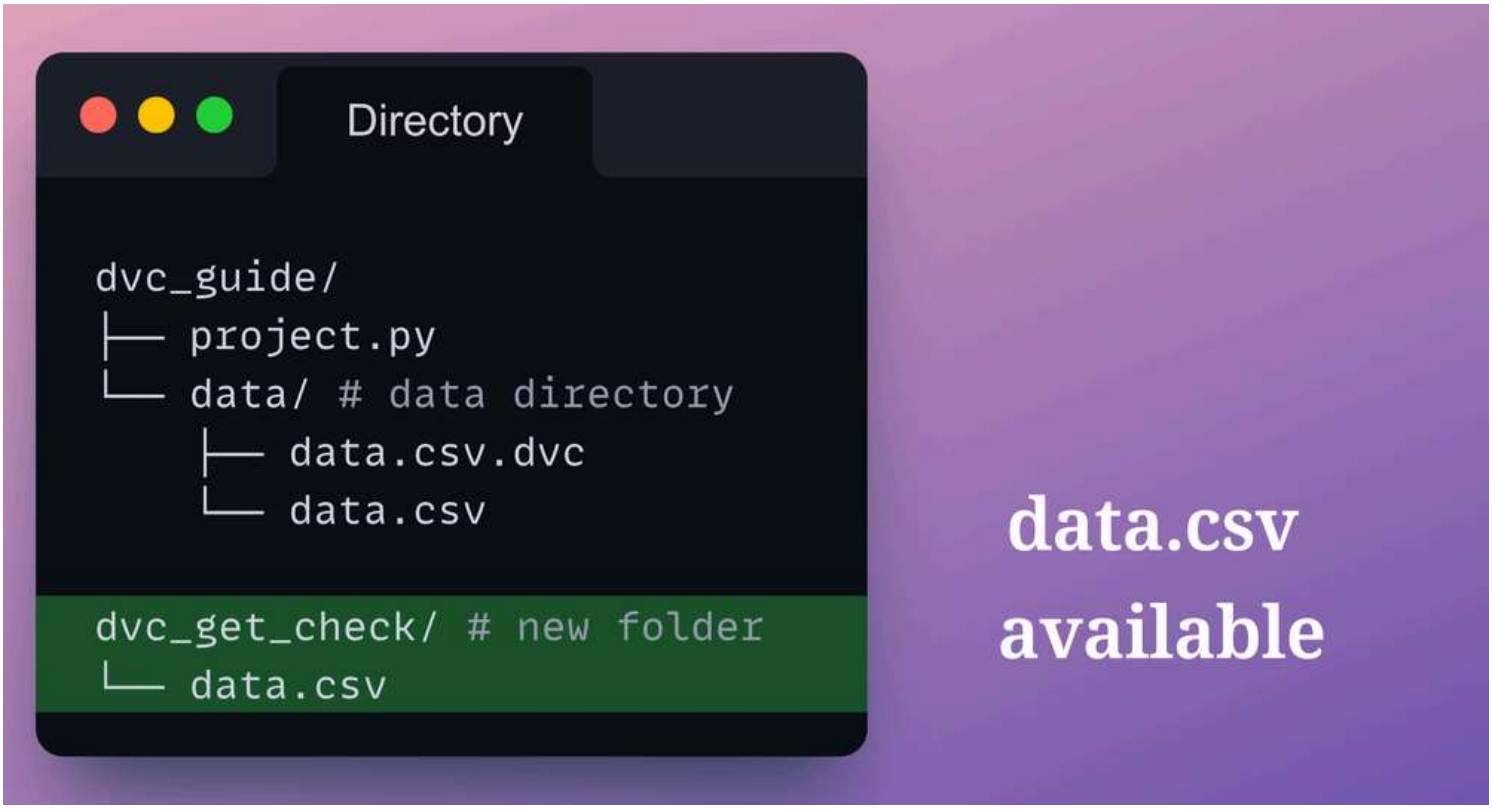
To recall, these are listed in the `.dvc/config` file:




When running `dvc get`, we must also tell the location (or alias) of the remote storage where the dataset is stored.

We do this using the `--remote` option of `dvc get`. The final command looks like:

After the download is complete, the `data.csv` file will be available in the new folder we created above.




# Download data file with source info

The above approach of `dvc get` lets us download a DVC-tracked dataset from cloud storage using the GitHub repository.

But the problem with this approach is that after downloading, the file just sits like any other file on our local machine.

In other words, `dvc get` does not maintain any record of where a specific file was downloaded from.

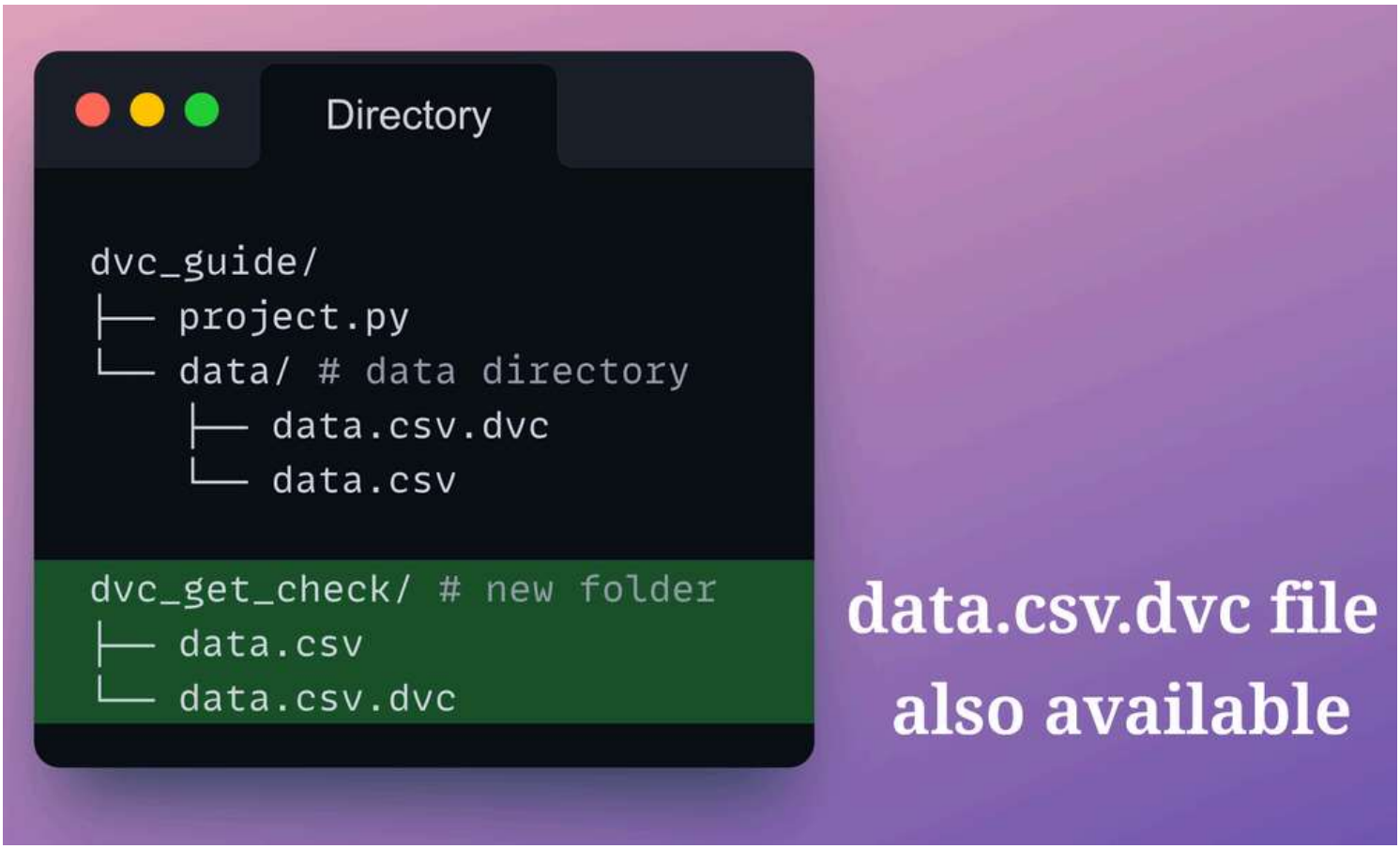
This can be important because let's say that after downloading the file, we revisit it after a few days.

At that time, we may need some additional details that are available in the source but we may not remember where we retrieved that dataset from.

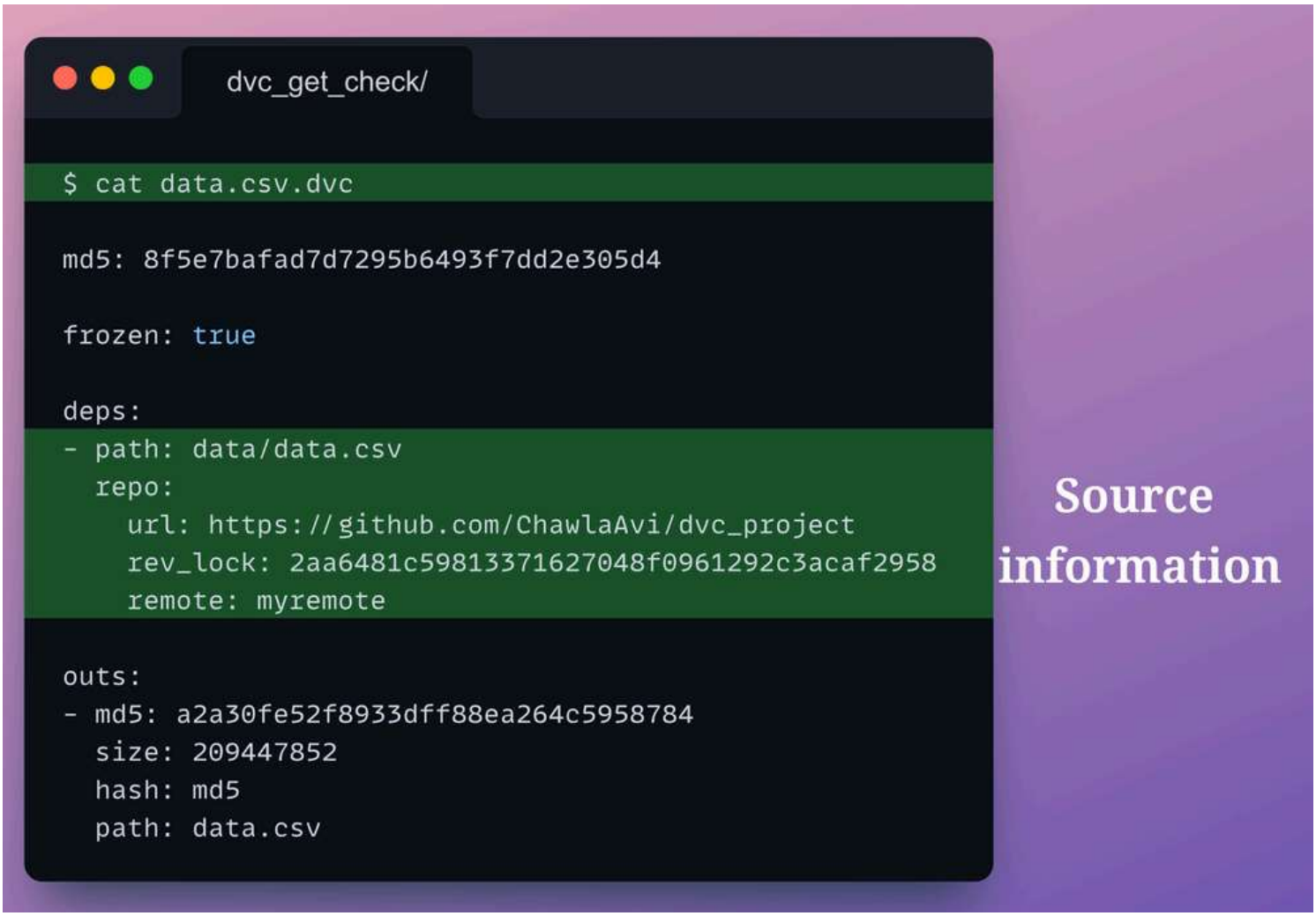
The `dvc import` command lets us resolve this issue.

The approach to downloading the dataset will be the same as what we did in `dvc get`.

This time, if we look at the project directory, we notice that DVC added a `data.csv.dvc` file as well:



Printing the contents of `data.csv.dvc` file, we see the following:




As depicted above, the hash file now also contains additional information about the file source, which was not available earlier.

Cool, isn't it?

# DVC self-project

To get a firm hold of this critical and must-know DVC skill, it is important to have some hands-on practice.

So, moving on, let's solidify our understanding of data version control using DVC by doing a small exercise.

The foundations that you need have already been covered in this entire article. If you have read everything so far, completing this exercise will be easy, fun, and a good way to reinforce those learnings through hands-on experimentation.

Download the zip file below.

**DVC-self-project**

DVC-self-project.zip • 46 MB

Open the `DVC-notebook.ipynb` file. This is where you shall code the entire exercise.

The objective is to replicate what we did in this article.

The entire notebook provides step-by-step instructions on what to do at a specific step and how to do it.

If you get stuck somewhere, `DVC-answer-notebook.ipynb` contains my solution.

You can download this notebook below and refer to it if you get stuck. **But I highly encourage you not to do that.**

**DVC-answer-notebook**

DVC-answer-notebook.ipynb • 1 MB

Try to do everything yourself.

Because if you successfully implement everything, I am sure it will give you plenty of confidence.

In case of any queries, feel free to reach out.

If possible, do share your solutions with me :)

# Conclusion

With this, we come to the end of this extensive deep dive on data version control using DVC.

Large dataset projects always have large datasets and models.

It is impossible to maintain a single record of truth on remote repositories like GitHub, which tracks everything from code to data to models.

Thus, it is quite necessary to have appropriate data version control systems as well, which can seamlessly integrate lightweight remote repositories with storage locations.

DVC lets us effortlessly version both the codebase and the large files of our project.

**But we must note that DVC is much more than what we discussed in this article.**

This article was primarily intended to introduce you to DVC.

In the upcoming deep dives, we shall get into more detail on building elegant and robust data pipelines with DVC.

Just like this deep dive, we'll do a hands-on tutorial on:

- How to build and execute machine learning pipelines with DVC

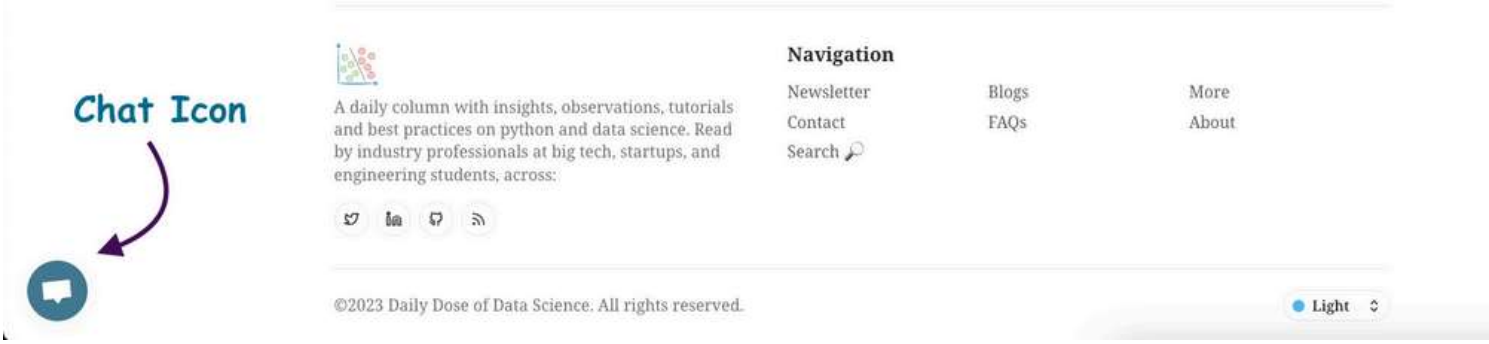
- How to cache the results of the intermediate stages of our pipelines with DVC for faster execution
- How to make our machine learning experimentation more organized and trackable with DVC.
- And a lot more.

Till then, if you have any questions...

Feel free to post them in the comments.

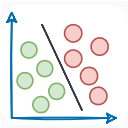
Or

If you wish to connect privately, feel free to initiate a chat here:




*Connect via chat*

Thanks for reading!



# Daily Dose of Data Science

A daily column with insights, observations, tutorials and best practices on python and data science. Read by industry professionals at big tech, startups, and engineering students.

Deployment Machine Learning MLOps



## Read next

LLMs Feb 18, 2024 • 47 min read

# A Beginner-friendly and Comprehensive Deep Dive on Vector Databases

Understanding every little detail on vector databases and their utility in LLMs, along with a hands-on demo.

Avi Chawla

Machine Learning Mar 12, 2024 • 22 min read

# A Detailed and Beginner-Friendly Introduction to PyTorch Lightning: The Supercharged PyTorch

Immensely simplify deep learning model building with PyTorch Lightning.

Avi Chawla

Deployment Oct 2, 2023 • 32 min read

# Sklearn Models are Not Deployment Friendly! Supercharge Them With Tensor Computations.

Speed up sklearn model inference up to 50x with GPU support.

Avi Chawla

Get Started!

A daily column with insights, observations, tutorials and best practices on python and data science. Read by industry professionals at big tech, startups, and engineering students, across:

## Navigation

Newsletter Blogs More Contact
FAQs About Search 🔎 Sign in
Get Started

 Light